LONG SHORT-TERM MEMORY (LSTM)

A Brief Introduction · XU Chentianye

OUTLINE

This presentation is divided into 6 sections:

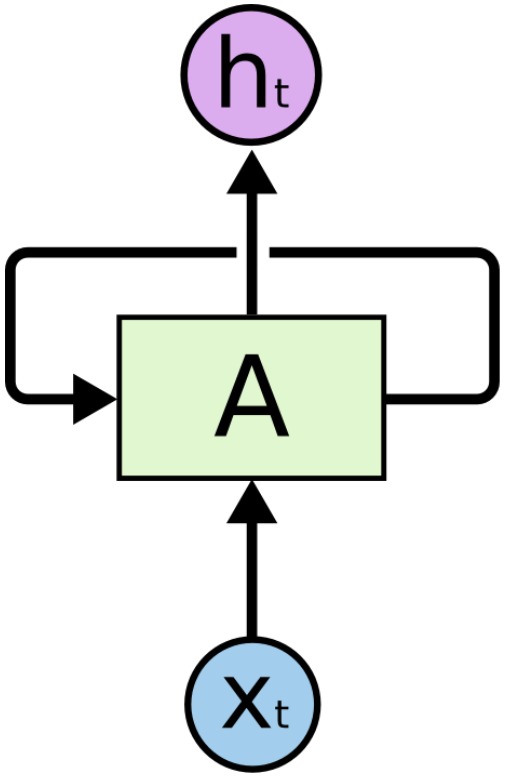
1. The Necessity: from RNN to LSTM
2. Structure and Function: Understand Core Idea
3. Variation: Gate Recurrent Unit (GRU)
4. Application 1: Stock Market Forecast
5. Application 2: Seq2seq
6. Conclusion and Discussion
7. References

OUTLINE

This presentation is divided into 6 sections:

1. The Necessity: from RNN to LSTM ☜
2. Structure and Function: Understand Core Idea
3. Variation: Gate Recurrent Unit (GRU)
4. Application 1: Stock Market Forecast
5. Application 2: Seq2seq
6. Conclusion and Discussion
7. References

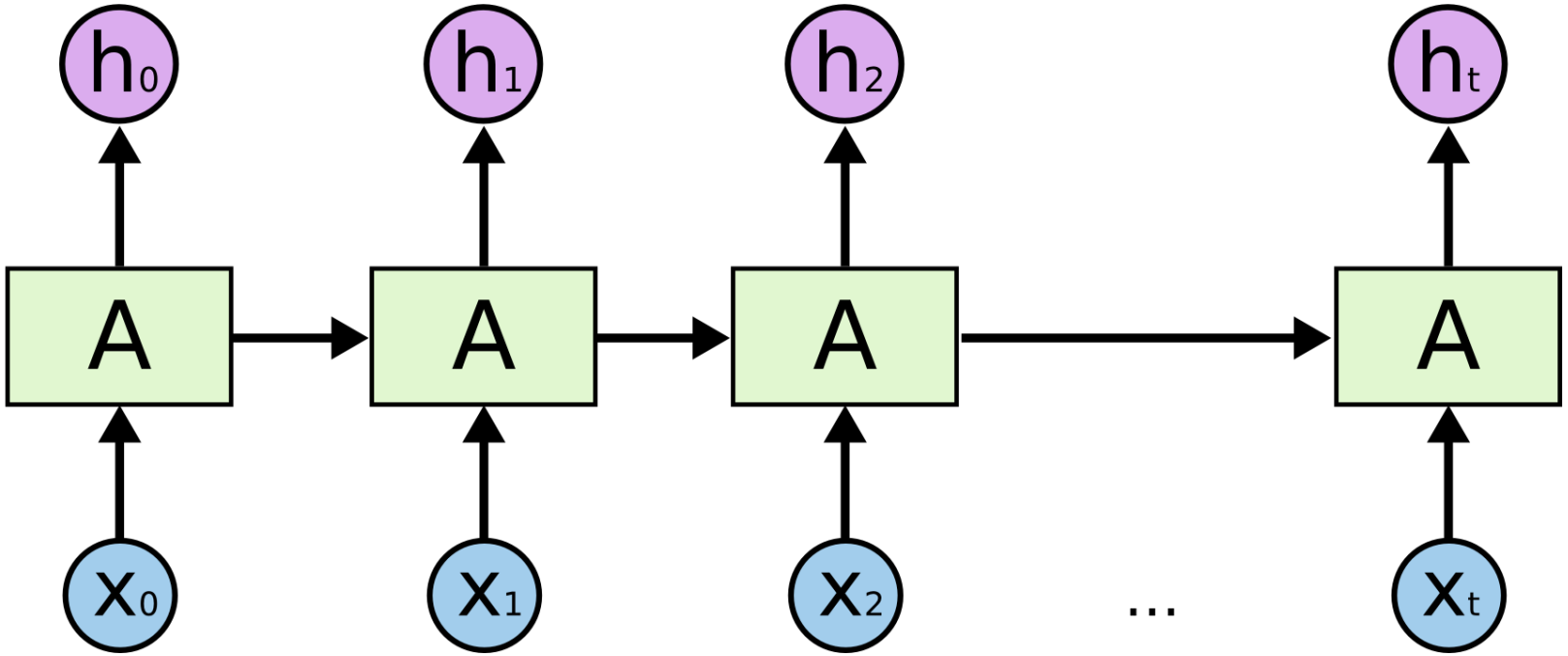
RECURRENT NEURAL NETWORK



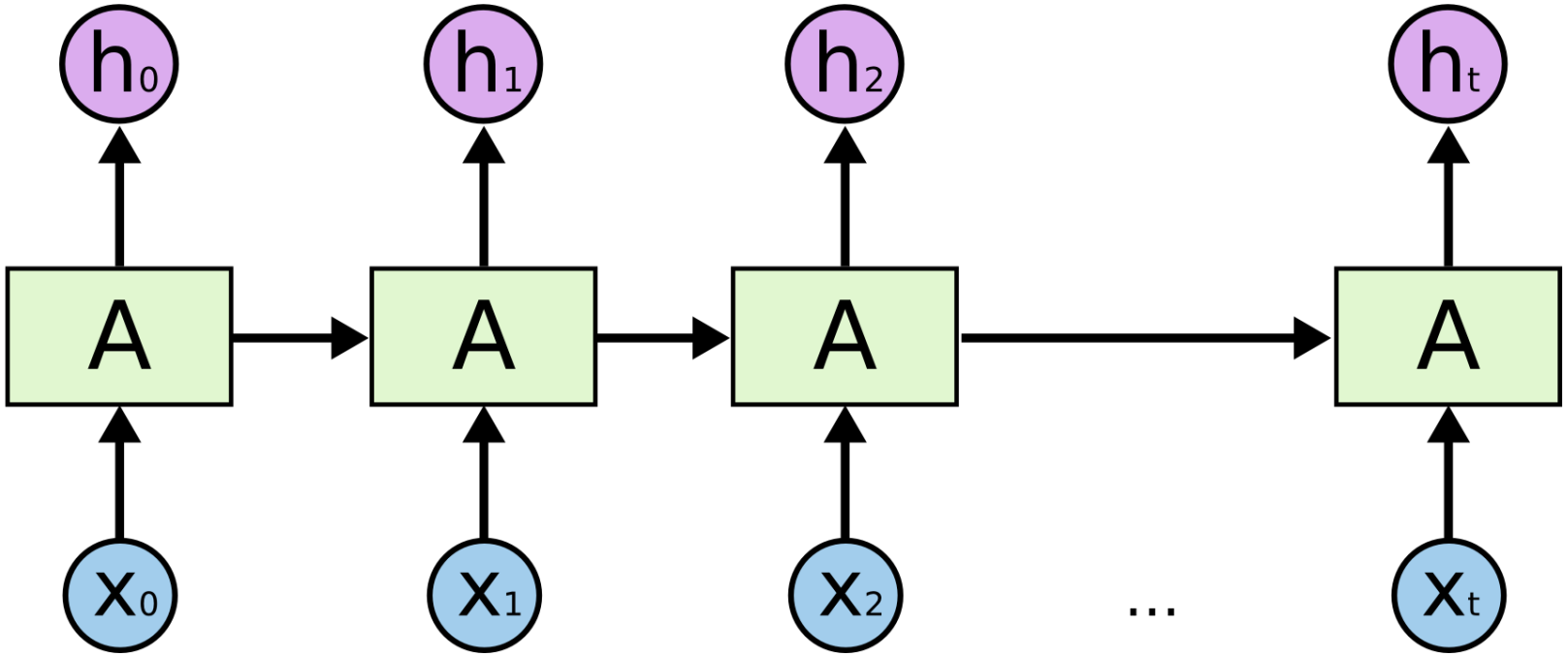
On the left is the typical structure of RNN:

a chunk of neural network, ***A***, looks at some input $\boldsymbol{x_t}$ and outputs a value $\boldsymbol{h_t}$. A loop allows information to be passed from one step of the network to the next.

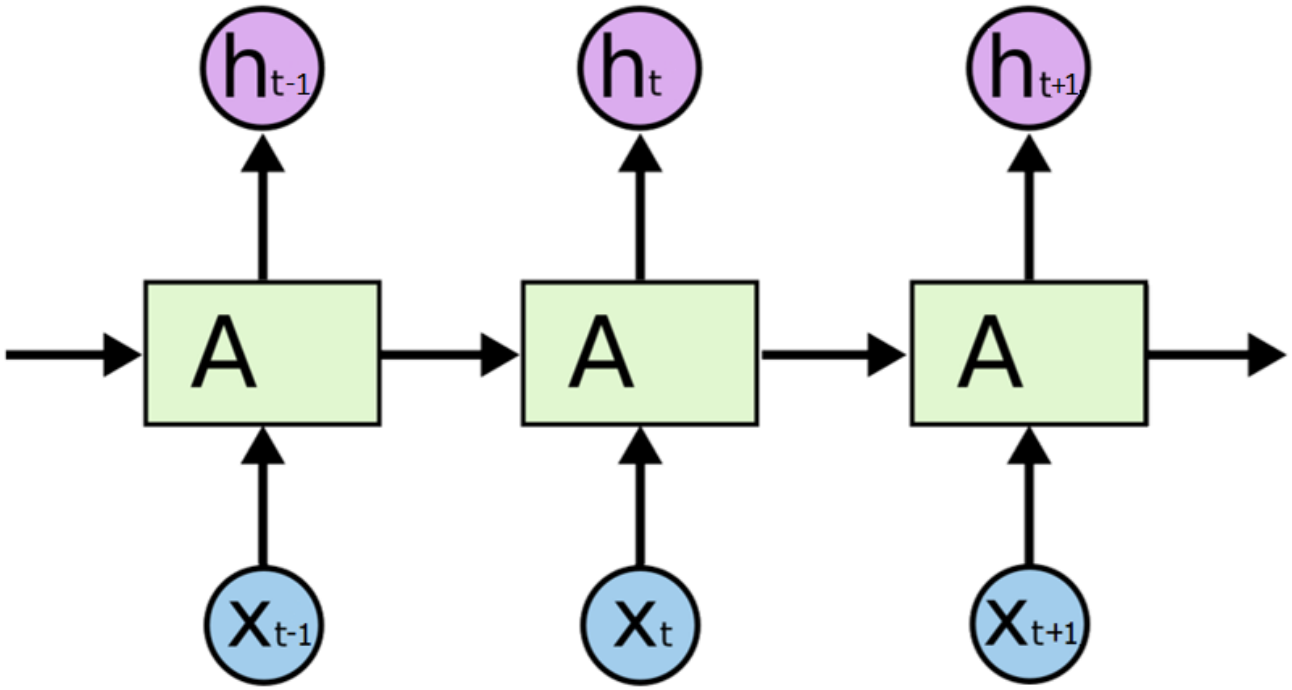
Unroll it…



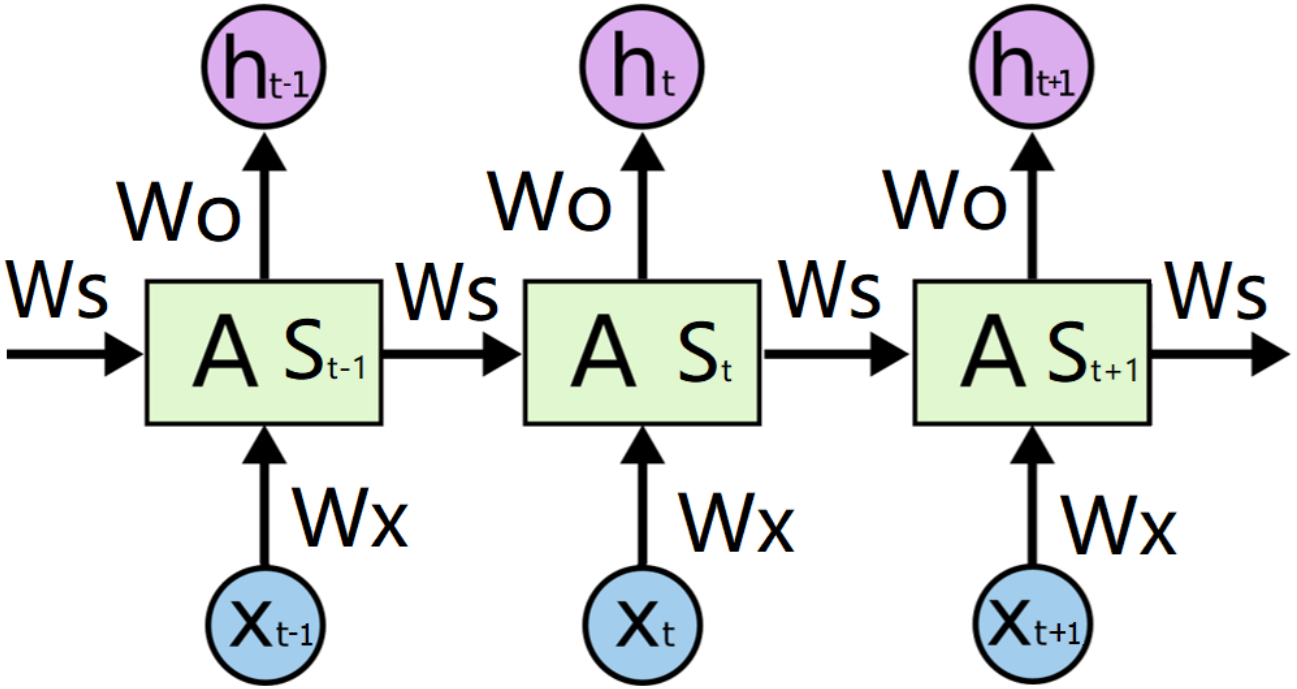
Unroll it…



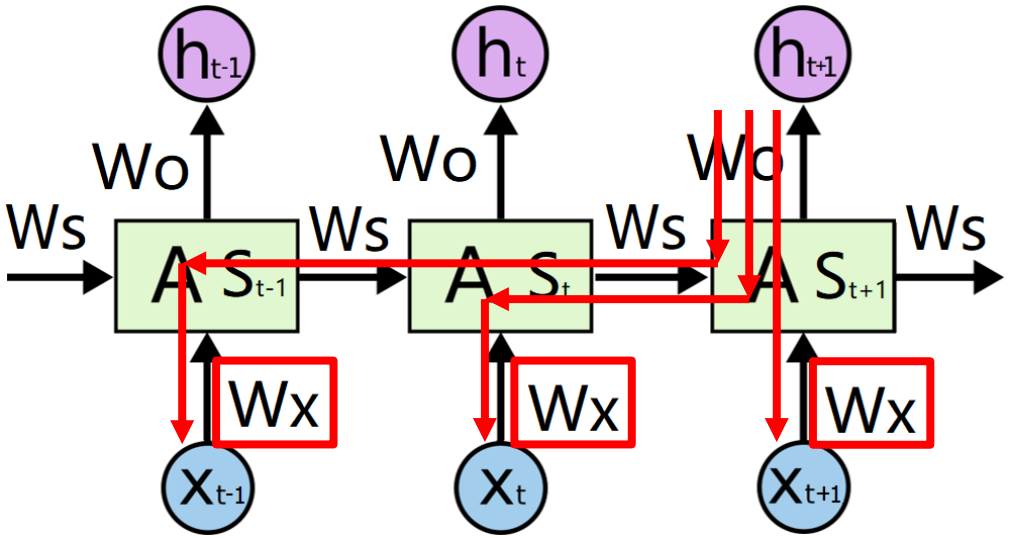
THE NECESSITY: Gradients of Weights in RNN



THE NECESSITY: Gradients of Weights in RNN



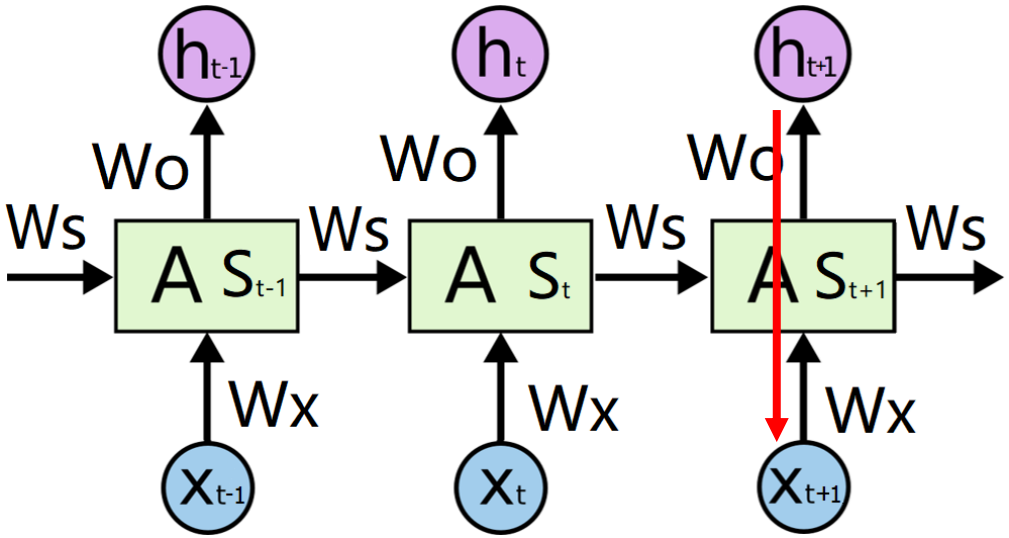
THE NECESSITY: Gradients of Weights in RNN



$$t+1=3$$

$$\frac{\partial L_3}{\partial W_x}=\frac{\partial L_3}{\partial W_x^{(3)}}+\frac{\partial L_3}{\partial W_x^{(2)}}+\frac{\partial L_3}{\partial W_x^{(1)}}$$

THE NECESSITY: Gradients of Weights in RNN

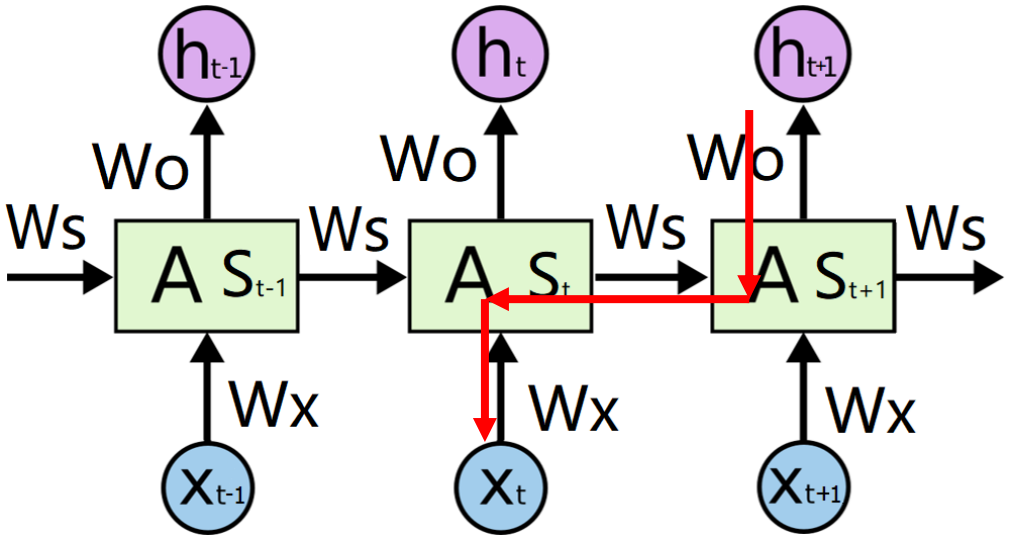


$$t+1=3$$

$$\frac{\partial L_3}{\partial W_x}=\boxed{\frac{\partial L_3}{\partial W_x^{(3)}}}+\frac{\partial L_3}{\partial W_x^{(2)}}+\frac{\partial L_3}{\partial W_x^{(1)}}$$

$$\frac{\partial L_3}{\partial W_x^{(3)}}=\frac{\partial L_3}{\partial h_3}\frac{\partial h_3}{\partial S_3}\frac{\partial S_3}{\partial W_x^{(3)}}$$

THE NECESSITY: Gradients of Weights in RNN

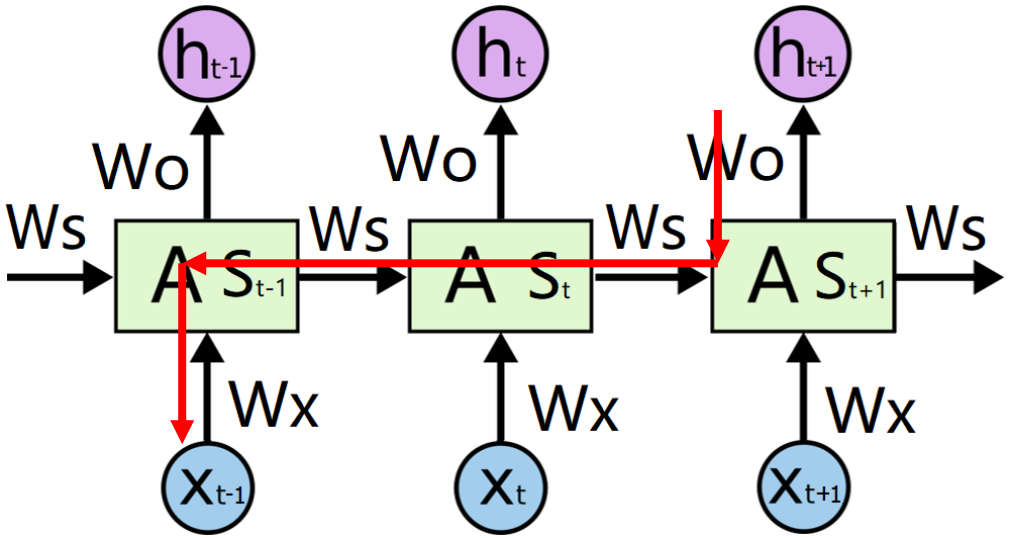


$$t+1=3$$

$$\frac{\partial L_3}{\partial W_x}=\frac{\partial L_3}{\partial W_x^{(3)}}+\boxed{\frac{\partial L_3}{\partial W_x^{(2)}}}+\frac{\partial L_3}{\partial W_x^{(1)}}$$

$$\frac{\partial L_3}{\partial W_x^{(2)}}=\frac{\partial L_3}{\partial h_3}\frac{\partial h_3}{\partial S_3}\frac{\partial S_3}{\partial S_2}\frac{\partial S_2}{\partial W_x^{(2)}}$$

THE NECESSITY: Gradients of Weights in RNN

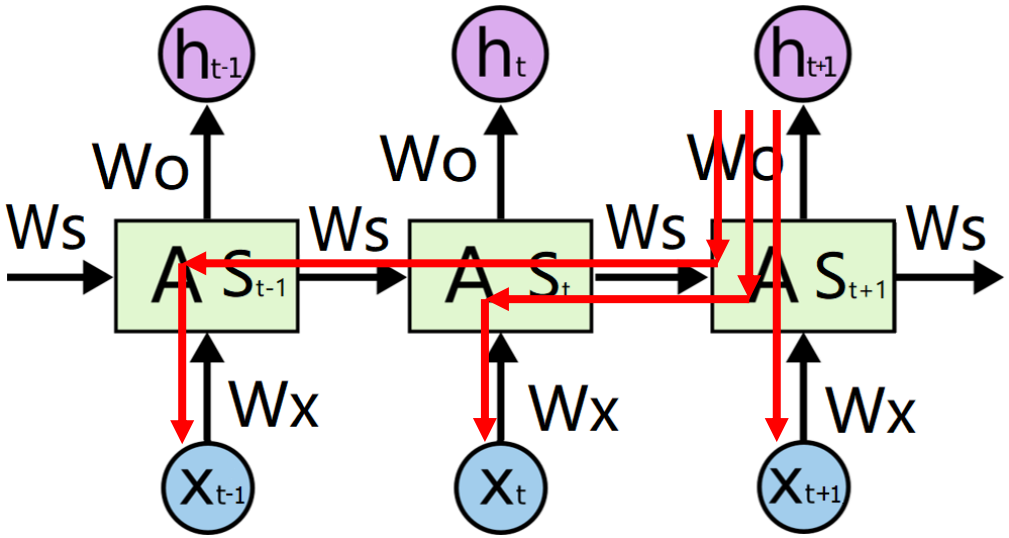


$$t+1=3$$

$$\frac{\partial L_3}{\partial W_x}=\frac{\partial L_3}{\partial W_x^{(3)}}+\frac{\partial L_3}{\partial W_x^{(2)}}+\boxed{\frac{\partial L_3}{\partial W_x^{(1)}}}$$

$$\frac{\partial L_3}{\partial W_x^{(1)}}=\frac{\partial L_3}{\partial h_3}\frac{\partial h_3}{\partial S_3}\frac{\partial S_3}{\partial S_2}\frac{\partial S_2}{\partial S_1}\frac{\partial S_1}{\partial W_x^{(1)}}$$

THE NECESSITY: Gradients of Weights in RNN



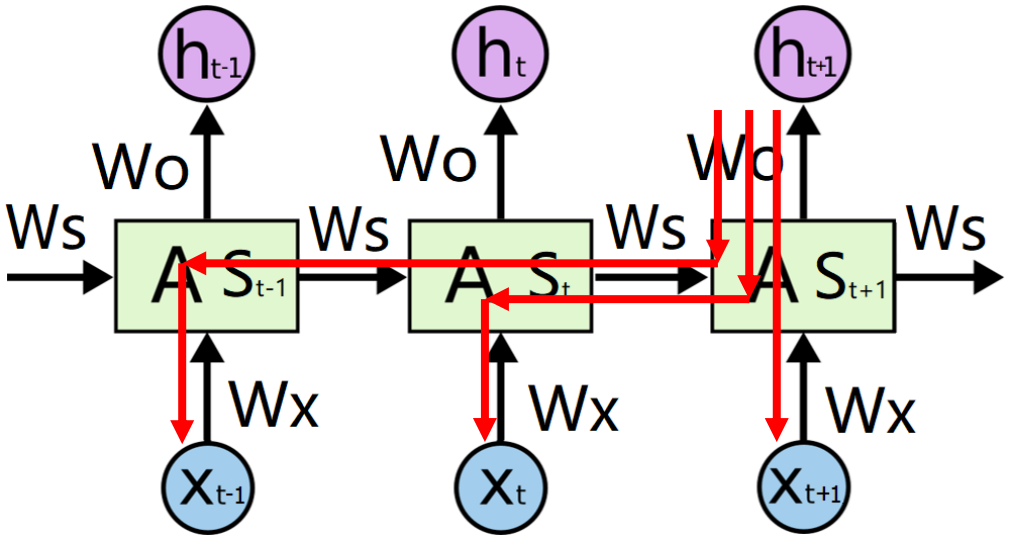
$$t+1=3$$

$$\frac{\partial L_3}{\partial W_x}=\frac{\partial L_3}{\partial W_x^{(3)}}+\frac{\partial L_3}{\partial W_x^{(2)}}+\frac{\partial L_3}{\partial W_x^{(1)}}$$

$$=\frac{\partial L_3}{\partial h_3}\frac{\partial h_3}{\partial S_3}\frac{\partial S_3}{\partial W_x^{(3)}}+$$

$$\frac{\partial L_3}{\partial h_3}\frac{\partial h_3}{\partial S_3}\left(\frac{\partial S_3}{\partial S_2}\frac{\partial S_2}{\partial W_x^{(2)}}+\frac{\partial S_3}{\partial S_2}\frac{\partial S_2}{\partial S_1}\frac{\partial S_1}{\partial W_x^{(1)}}\right)$$

THE NECESSITY: Gradients of Weights in RNN



$$t+1=3$$

$$\frac{\partial L_3}{\partial W_x}=\frac{\partial L_3}{\partial W_x^{(3)}}+\frac{\partial L_3}{\partial W_x^{(2)}}+\frac{\partial L_3}{\partial W_x^{(1)}}$$

$$=\frac{\partial L_3}{\partial h_3}\frac{\partial h_3}{\partial S_3}\frac{\partial S_3}{\partial W_x^{(3)}}+$$

$$\sum_{k=1}^{2}\frac{\partial L_3}{\partial h_3}\frac{\partial h_3}{\partial S_3}\left(\prod_{j=k+1}^{3}\frac{\partial S_j}{\partial S_{j-1}}\right)\frac{\partial S_k}{\partial W_x^{(k)}}$$

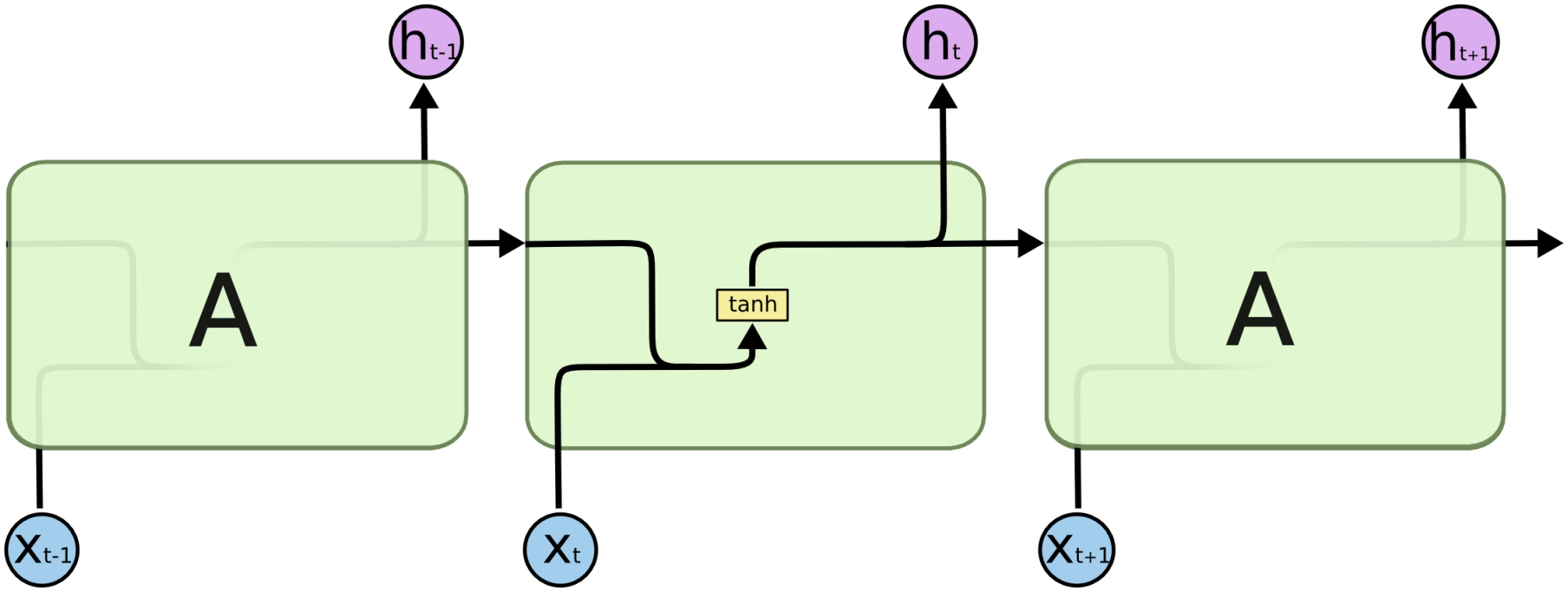
THE NECESSITY: Gradients of Weights in RNN

$$\frac{\partial L_t}{\partial W_x} = \frac{\partial L_t}{\partial W_x^{(t)}} + \ldots + \frac{\partial L_t}{\partial W_x^{(2)}} + \frac{\partial L_t}{\partial W_x^{(1)}}$$

$$= \frac{\partial L_t}{\partial h_t} \frac{\partial h_t}{\partial S_t} \frac{\partial S_t}{\partial W_x^{(t)}} +$$

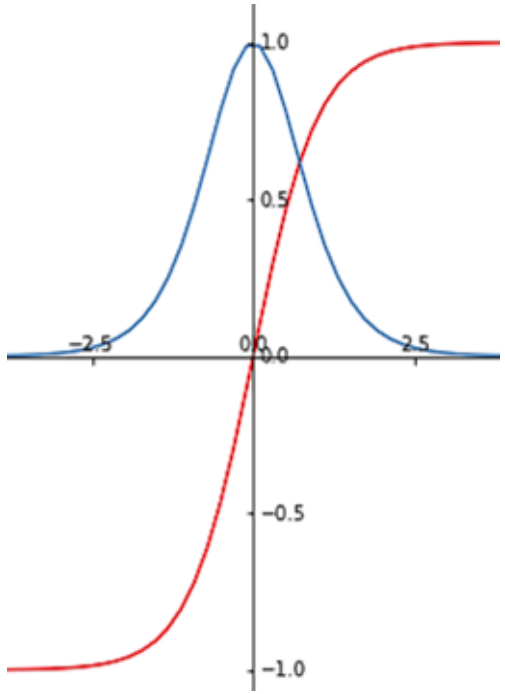
$$\sum_{k=1}^{t-1} \frac{\partial L_t}{\partial h_t} \frac{\partial h_t}{\partial S_t} \left(\prod_{j=k+1}^{t} \frac{\partial S_j}{\partial S_{j-1}} \right) \frac{\partial S_k}{\partial W_x^{(k)}}$$

THE NECESSITY: Gradients of Weights in RNN



$$\frac{\partial S_j}{\partial S_{j-1}} = \tanh'(\theta_j) W_s$$

THE NECESSITY: Gradients of Weights in RNN



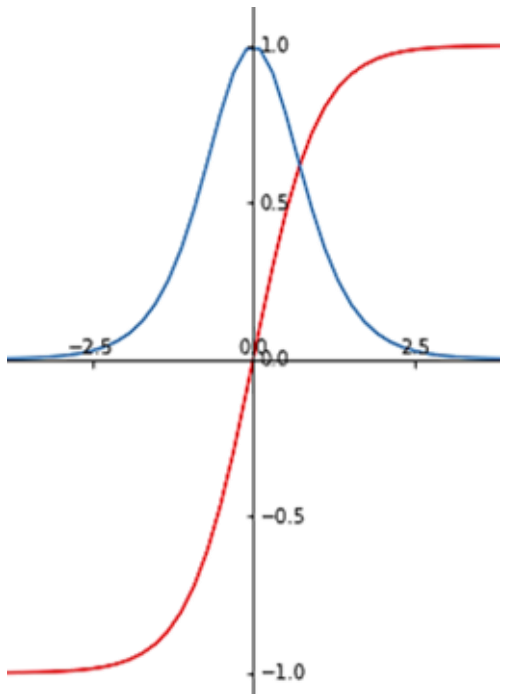
$$\frac{\partial L_t}{\partial W_x} = \frac{\partial L_t}{\partial W_x^{(t)}} + \ldots + \frac{\partial L_t}{\partial W_x^{(2)}} + \frac{\partial L_t}{\partial W_x^{(1)}}$$

$$= \frac{\partial L_t}{\partial h_t}\frac{\partial h_t}{\partial S_t}\frac{\partial S_t}{\partial W_x^{(t)}} +$$

$$\sum_{k=1}^{t-1}\frac{\partial L_t}{\partial h_t}\frac{\partial h_t}{\partial S_t}\left(\boxed{\prod_{j=k+1}^{t}\frac{\partial S_j}{\partial S_{j-1}}}\right)\frac{\partial S_k}{\partial W_x^{(k)}}$$

where $\boxed{\frac{\partial S_j}{\partial S_{j-1}} = \tanh'(\theta_j)W_s}$

THE NECESSITY: Gradients of Weights in RNN



$$\frac{\partial L_t}{\partial W_x} = \frac{\partial L_t}{\partial W_x^{(t)}} + \ldots + \frac{\partial L_t}{\partial W_x^{(2)}} + \frac{\partial L_t}{\partial W_x^{(1)}}$$

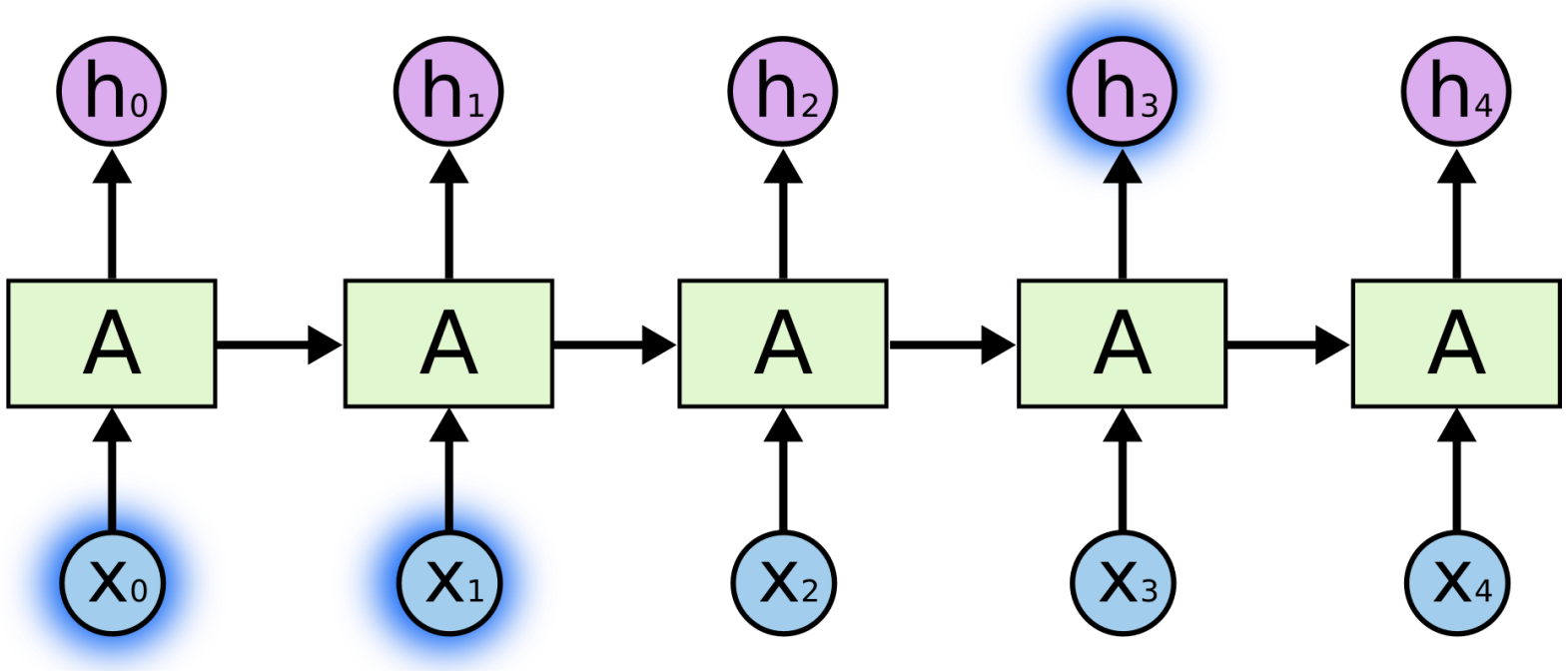
$$= \frac{\partial L_t}{\partial h_t}\frac{\partial h_t}{\partial S_t}\frac{\partial S_t}{\partial W_x^{(t)}} +$$

$$\sum_{k=1}^{t-1}\frac{\partial L_t}{\partial h_t}\frac{\partial h_t}{\partial S_t}\left(\boxed{\prod_{j=k+1}^{t}\frac{\partial S_j}{\partial S_{j-1}}}\right)\frac{\partial S_k}{\partial W_x^{(k)}}$$

where $\boxed{\frac{\partial S_j}{\partial S_{j-1}} = \tanh'(\theta_j)W_s}$

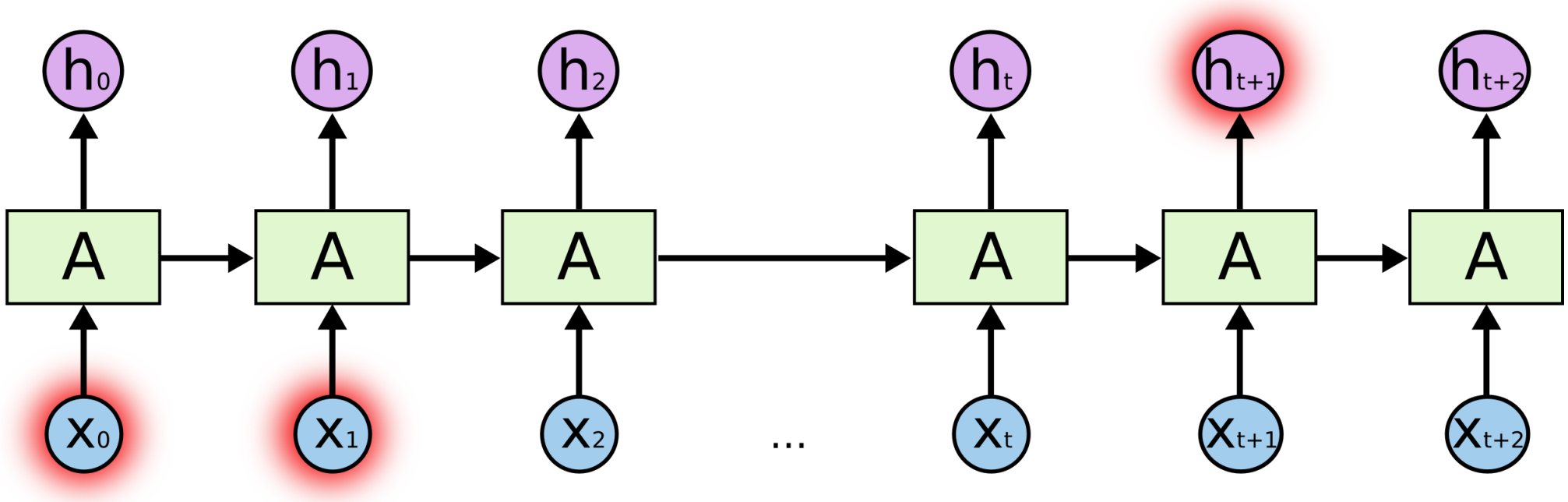
THE NECESSITY: Gradients of Weights in RNN

The clouds are in the sky.



THE NECESSITY: Gradients of Weights in RNN

… here for several days of rain, so the air is very fresh.



POSSIBLE SOLUTIONS

1. Alter activation functions. (i.e. ReLU)
2. Add batch normalization layer.
3. Recast the structure of RNN. (i.e. LSTM)

OUTLINE

This presentation is divided into 6 sections:

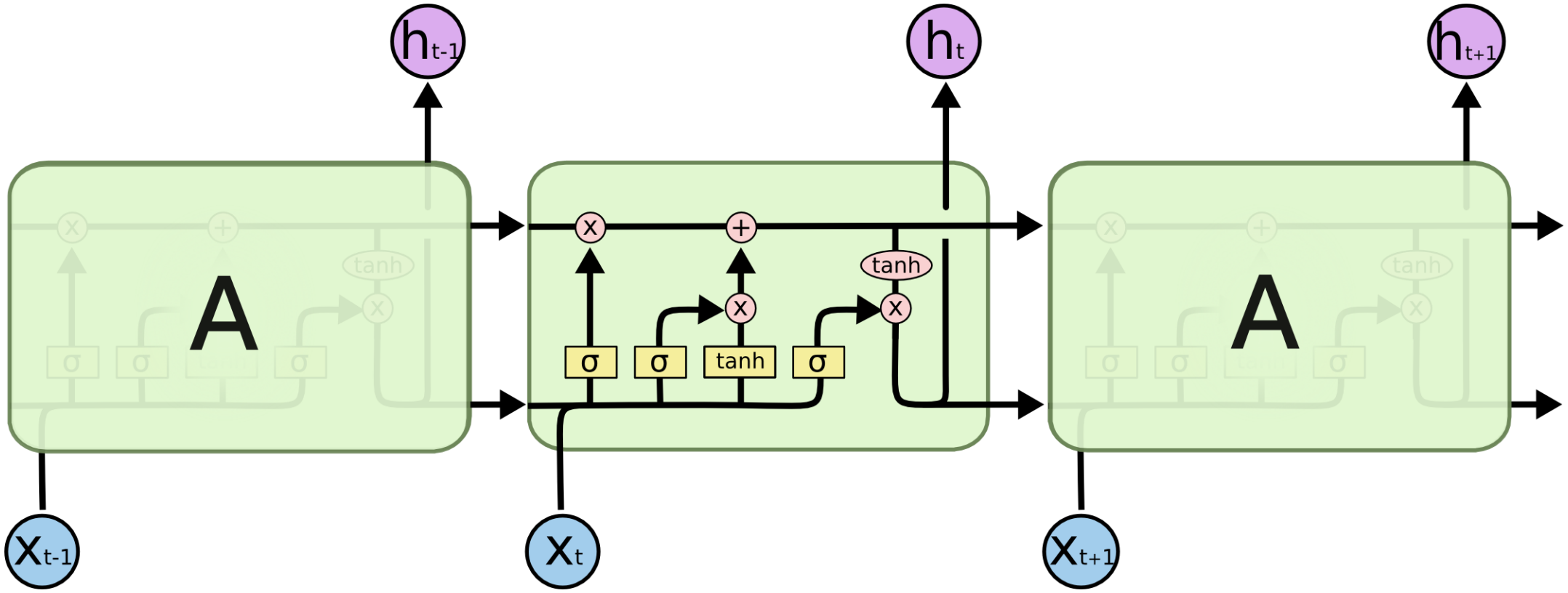
1. The Necessity: from RNN to LSTM
2. Structure and Function: Understand Core Idea ☜
3. Variation: Gate Recurrent Unit (GRU)
4. Application 1: Stock Market Forecast
5. Application 2: Seq2seq
6. Conclusion and Discussion
7. References

A LITTLE BACKGROUND ON LSTM

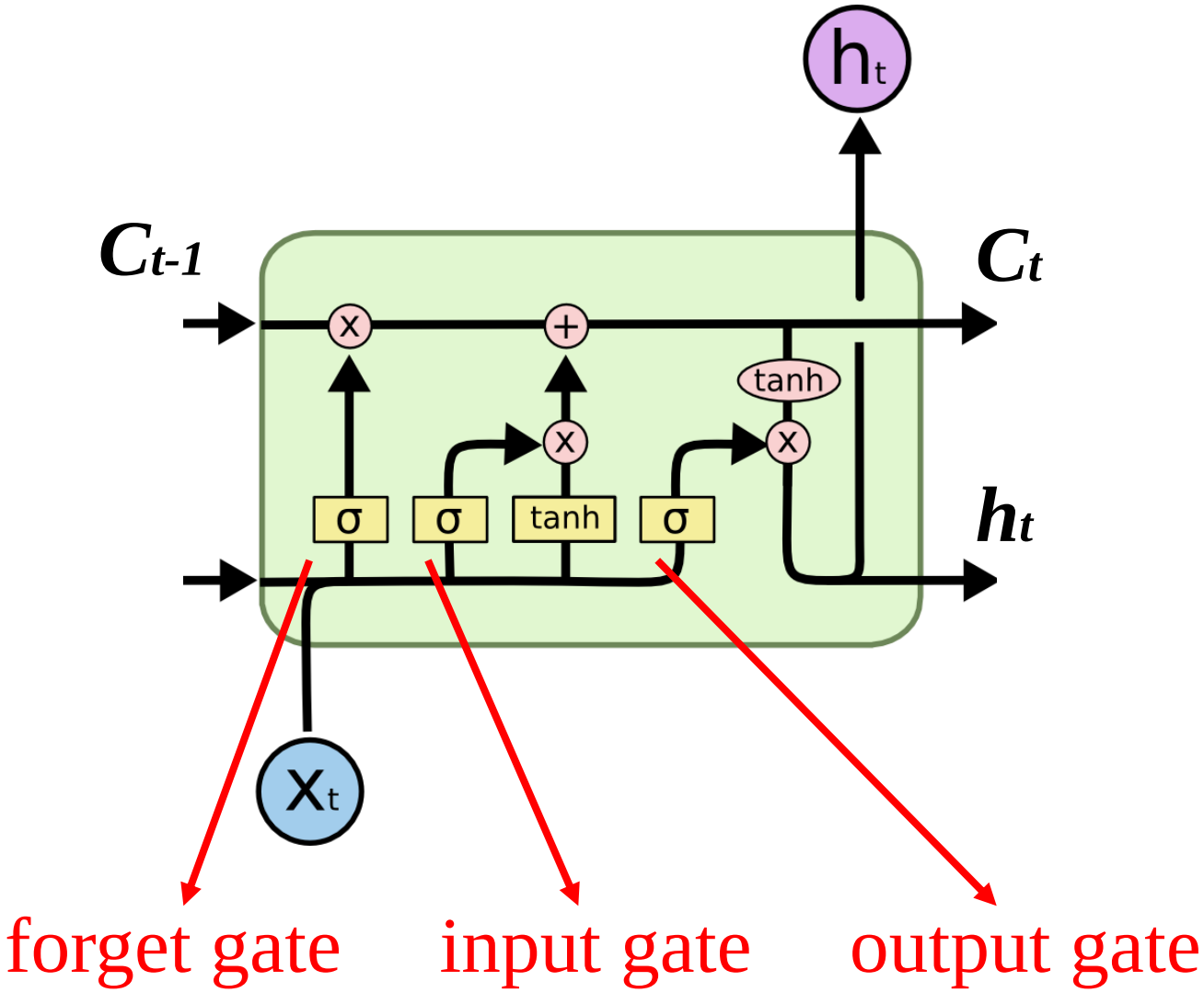
Sepp Hochreiter

Jürgen Schmidhuber

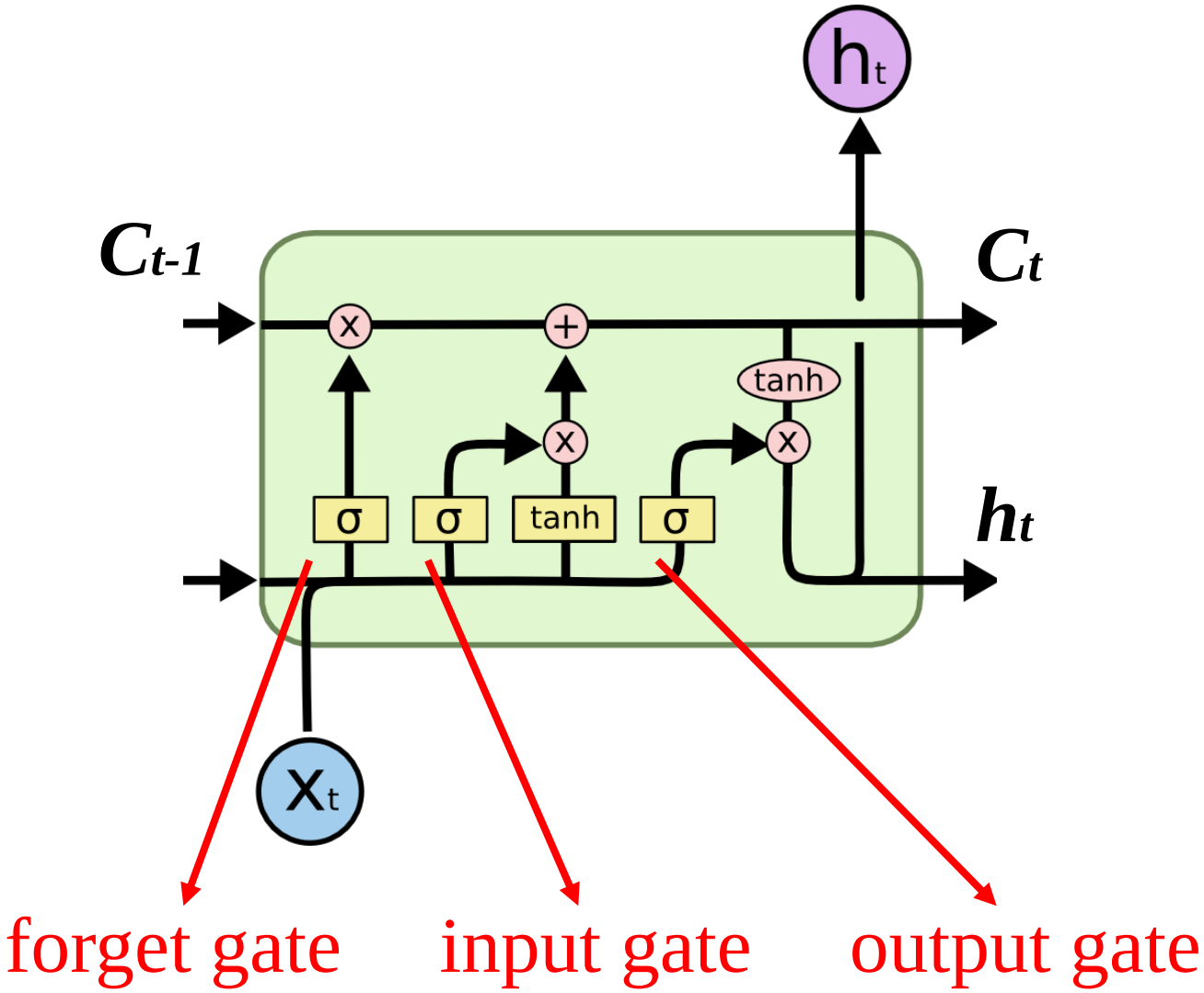
A LITTLE BACKGROUND ON LSTM



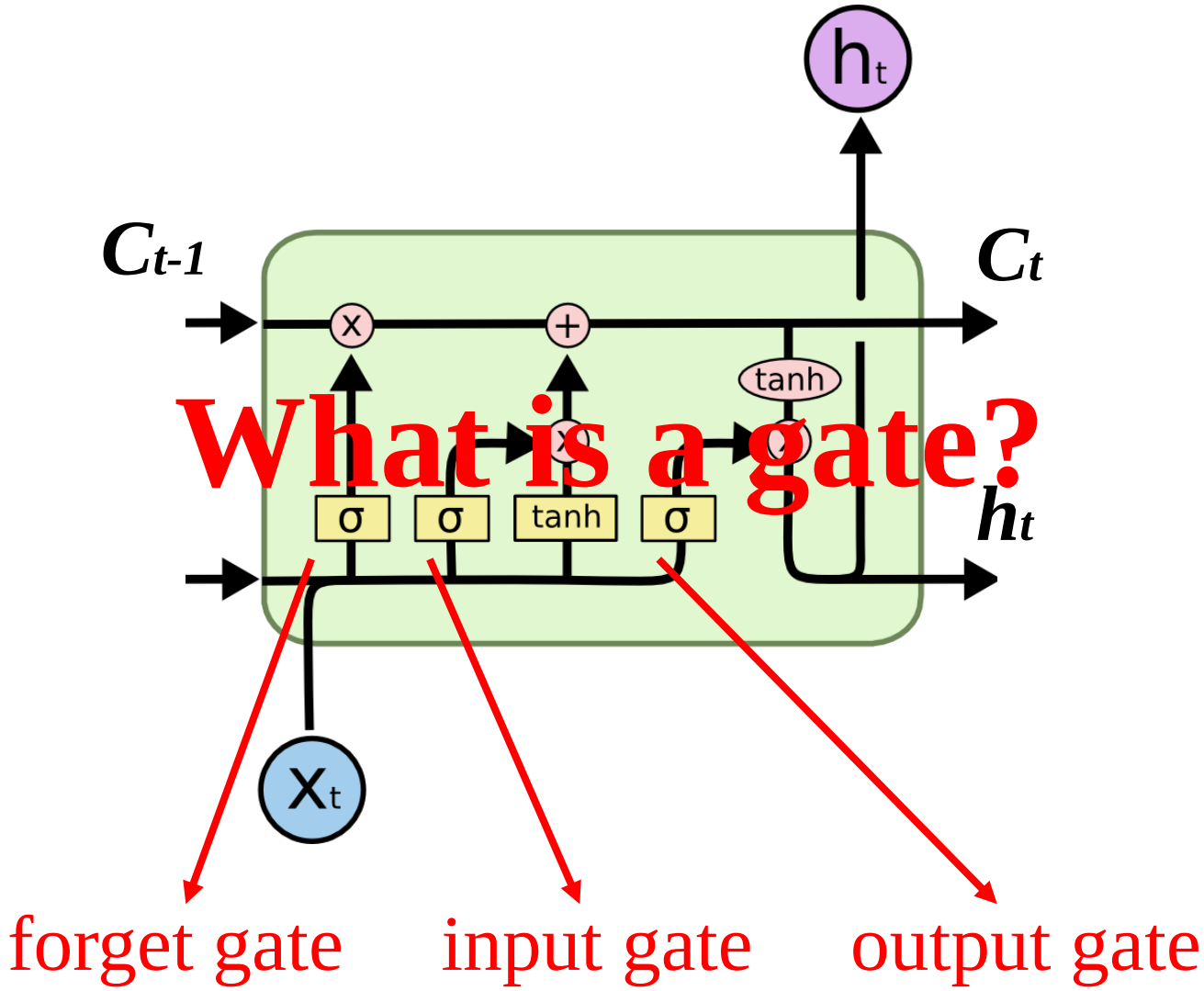
A LITTLE BACKGROUND ON LSTM



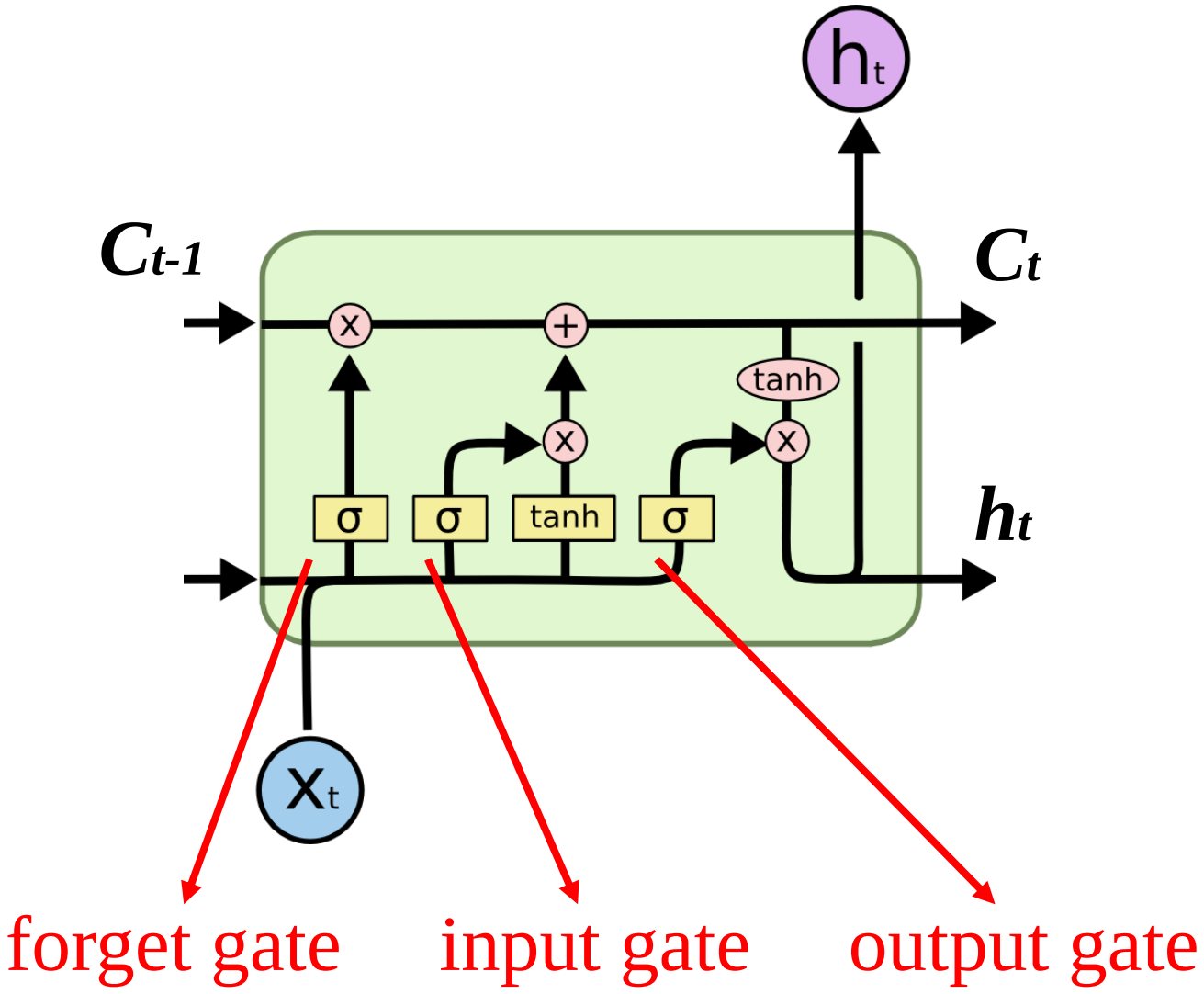
A LITTLE BACKGROUND ON LSTM

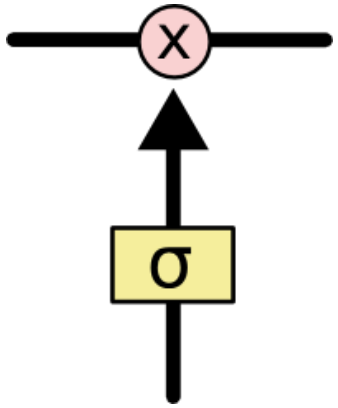


ON STRUCTURE: Analyze Functions of Gates



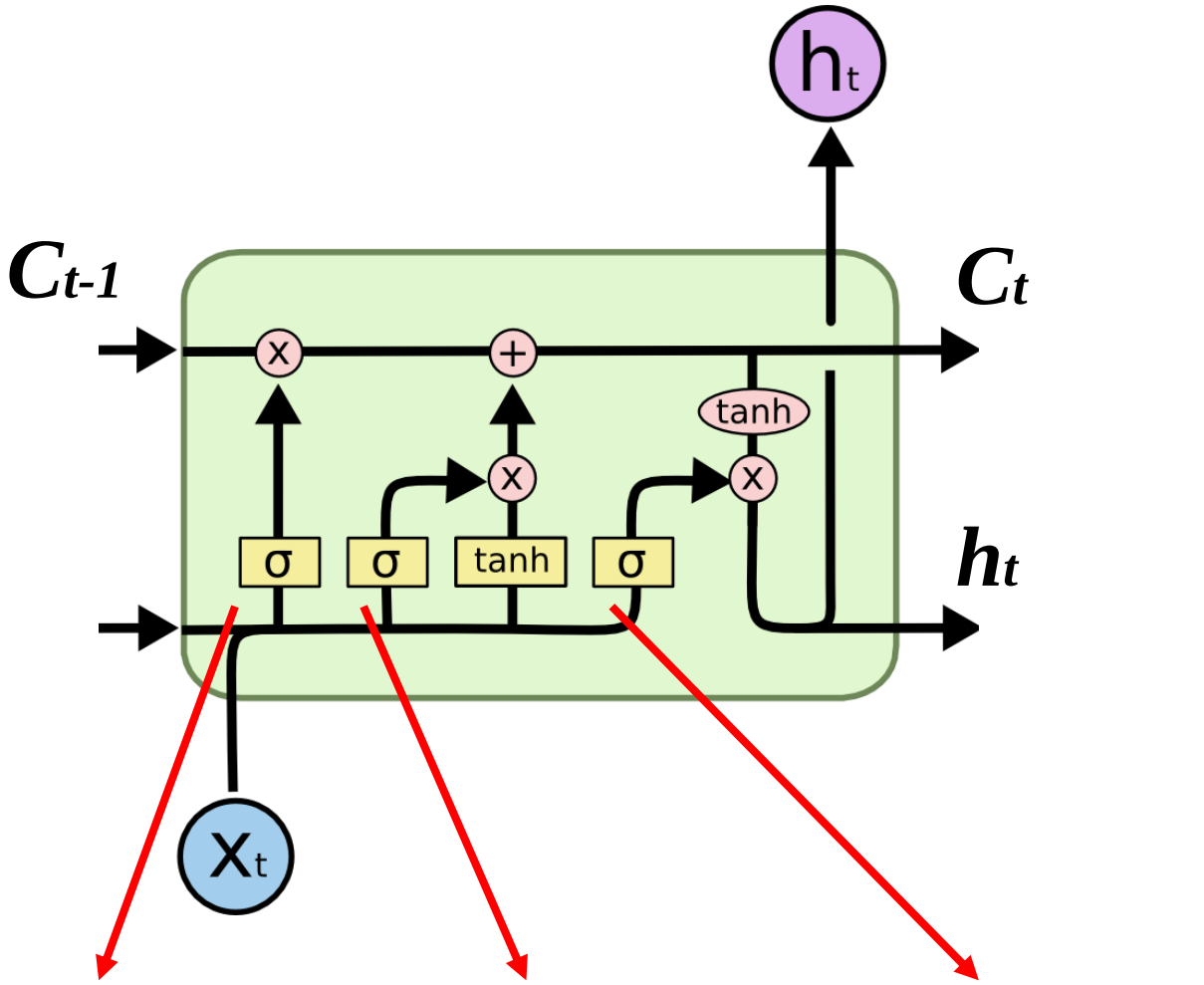
ON STRUCTURE: Analyze Functions of Gates

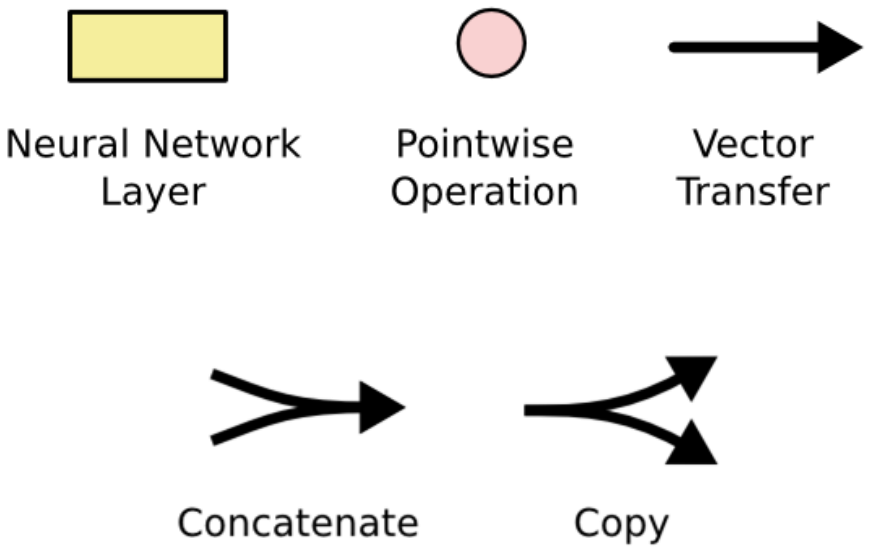




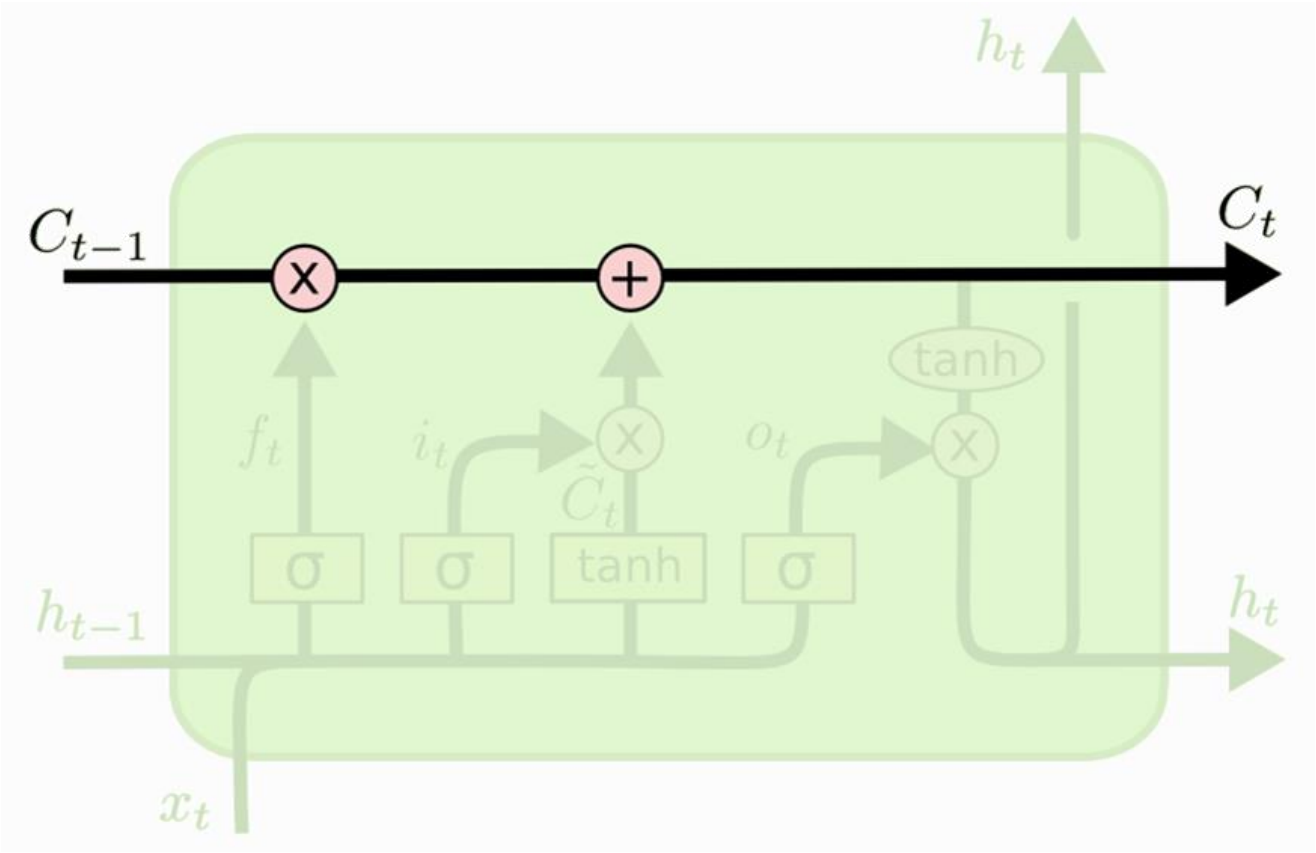
A gate

ON STRUCTURE: Analyze Functions of Gates

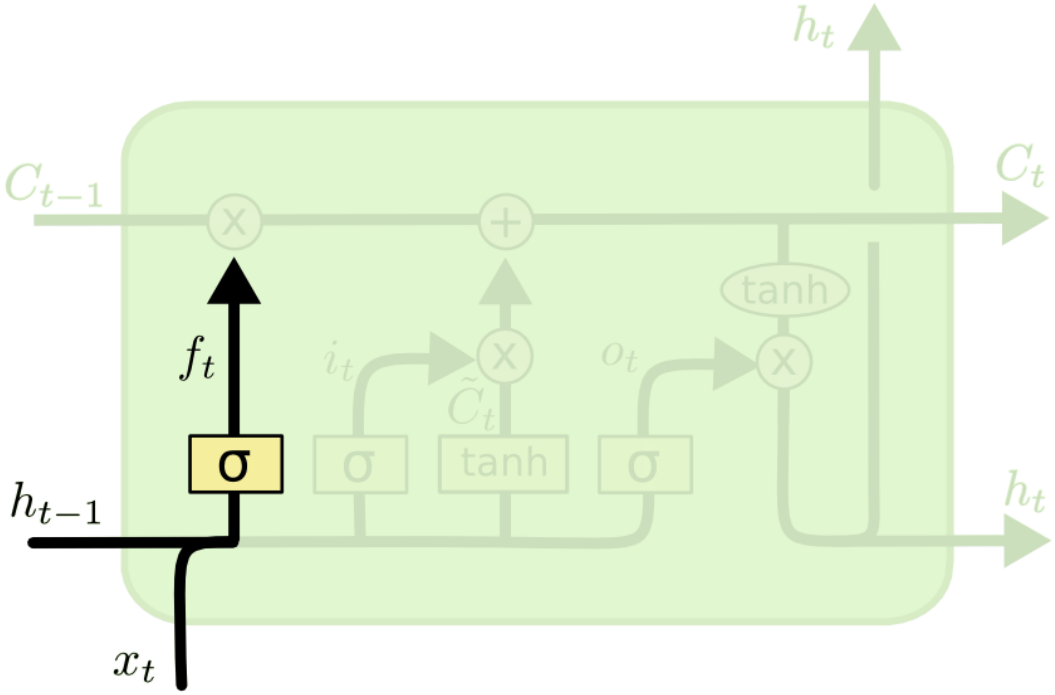




CORE IDEAS BEHIND LSTM: Cell State

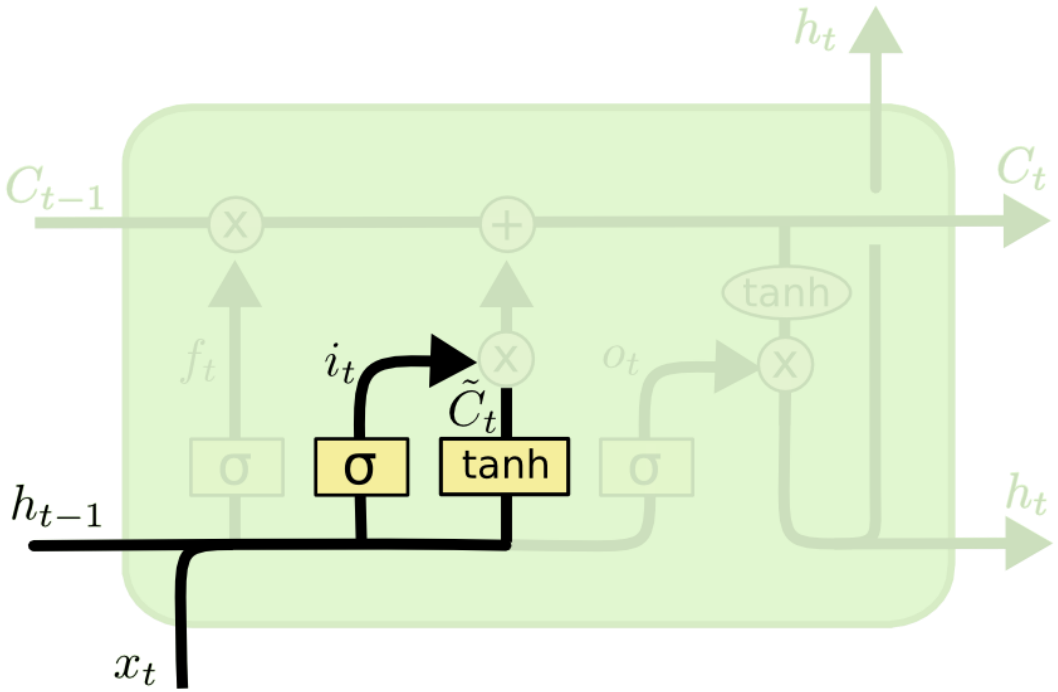


STEP-BY-STEP LSTM WALK THROUGH: Forget Gate



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \ + \ b_f\right)$$

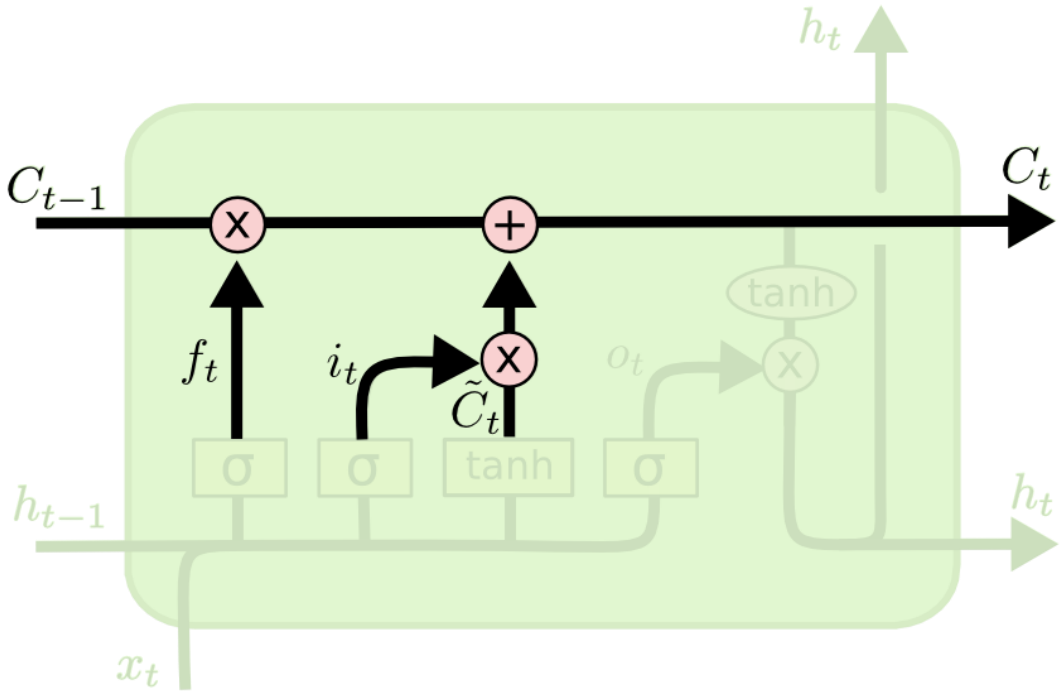
STEP-BY-STEP LSTM WALK THROUGH: Input Gate



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \ + \ b_i\right)$$

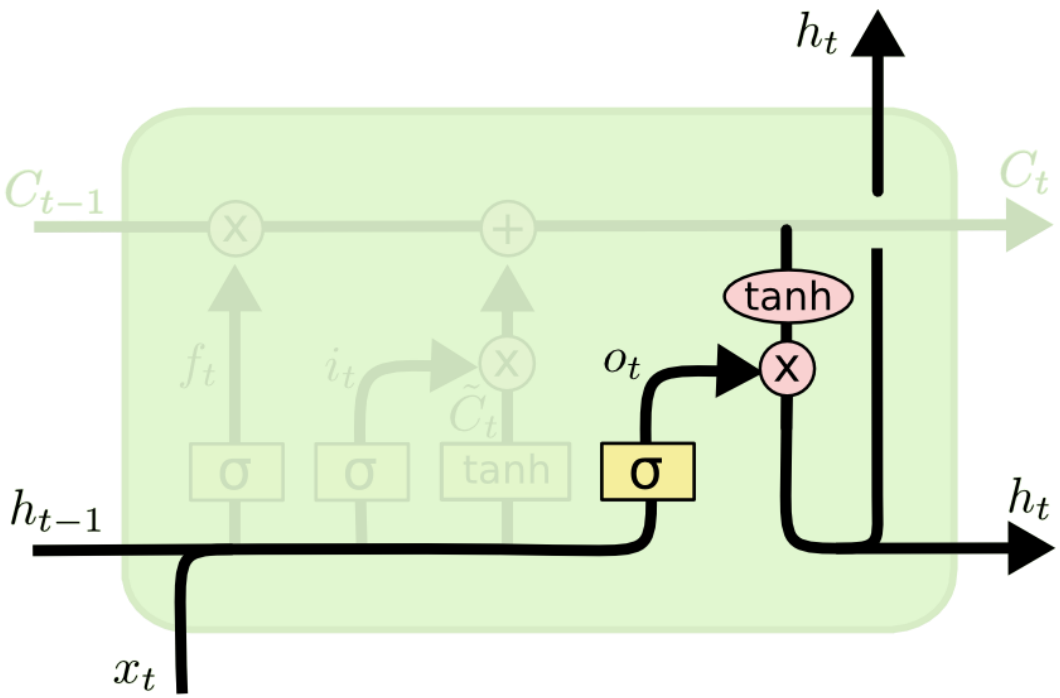
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \ + \ b_C)$$

STEP-BY-STEP LSTM WALK THROUGH: Input Gate



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

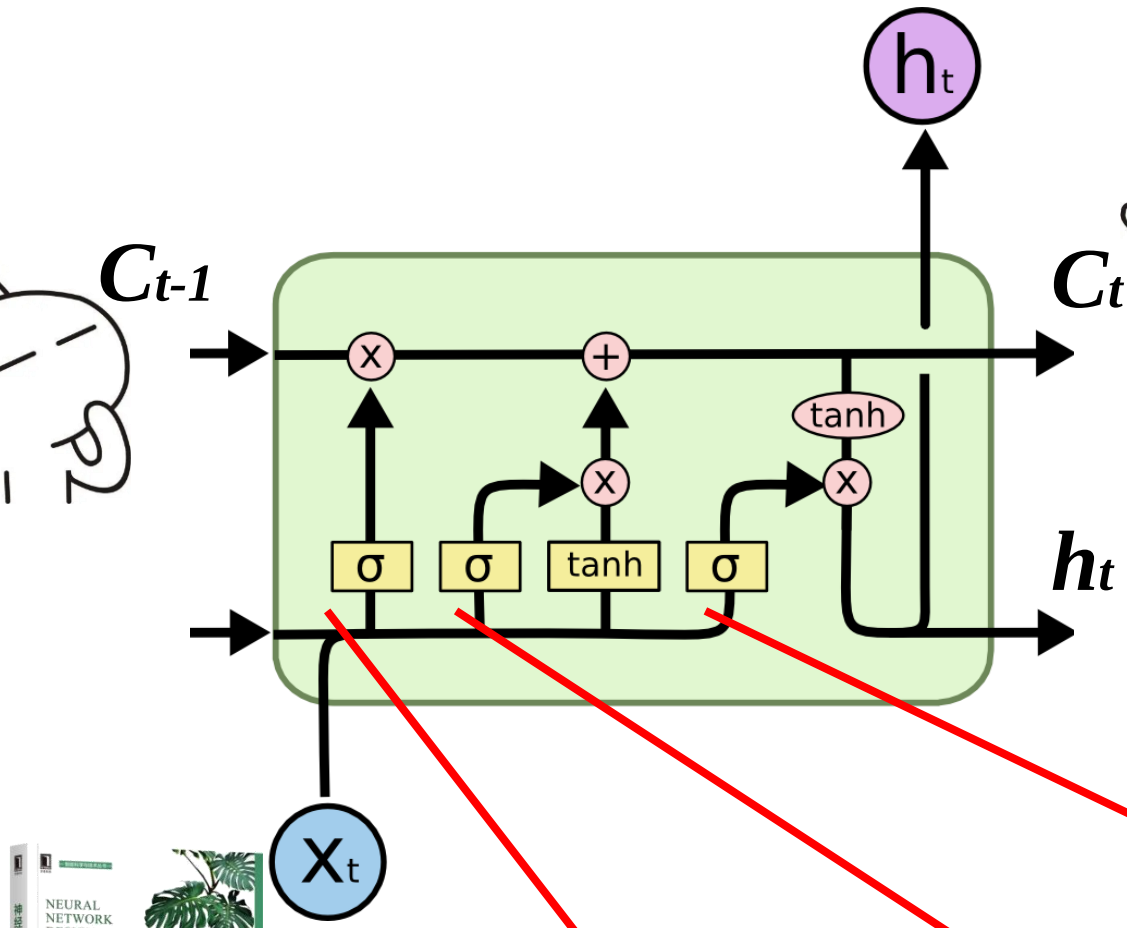
STEP-BY-STEP LSTM WALK THROUGH: Input Gate

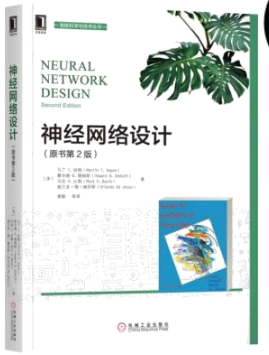


$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

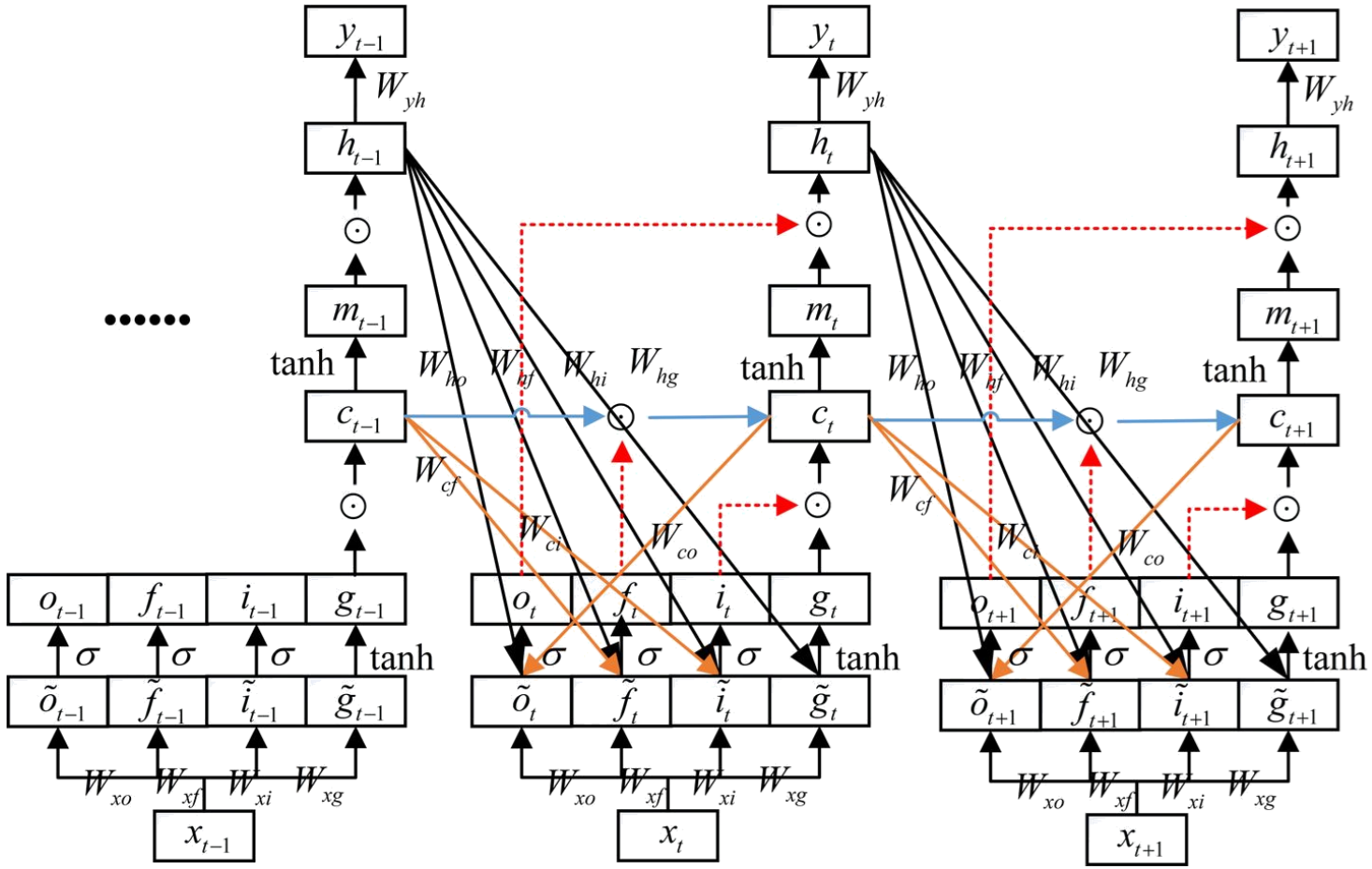
$$h_t = o_t * \tanh\left(C_t\right)$$

AN EXAMPLE: Taking Final Exams



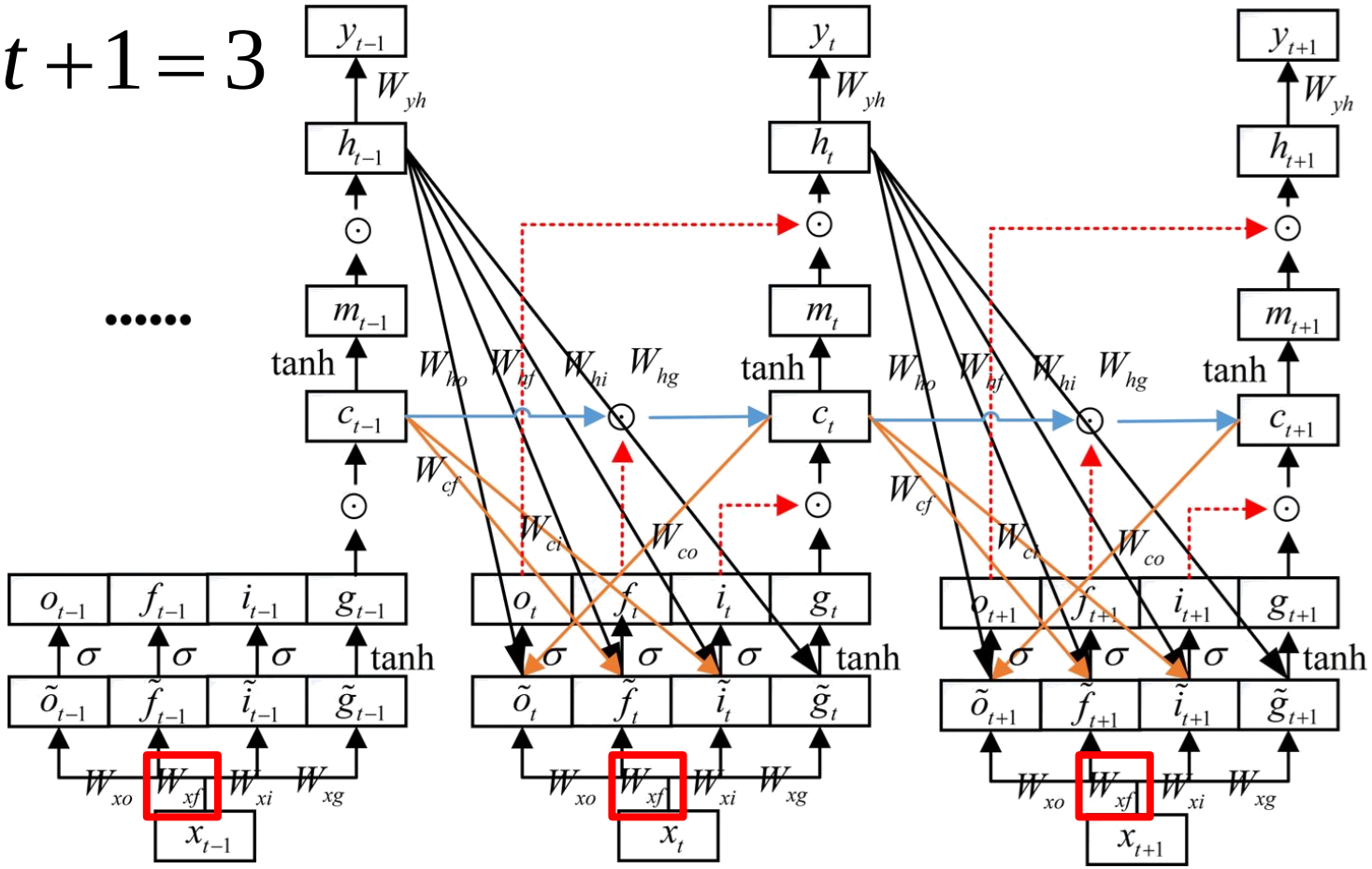


EXPLORATION: How LSTM Solves Vanishing Gradients



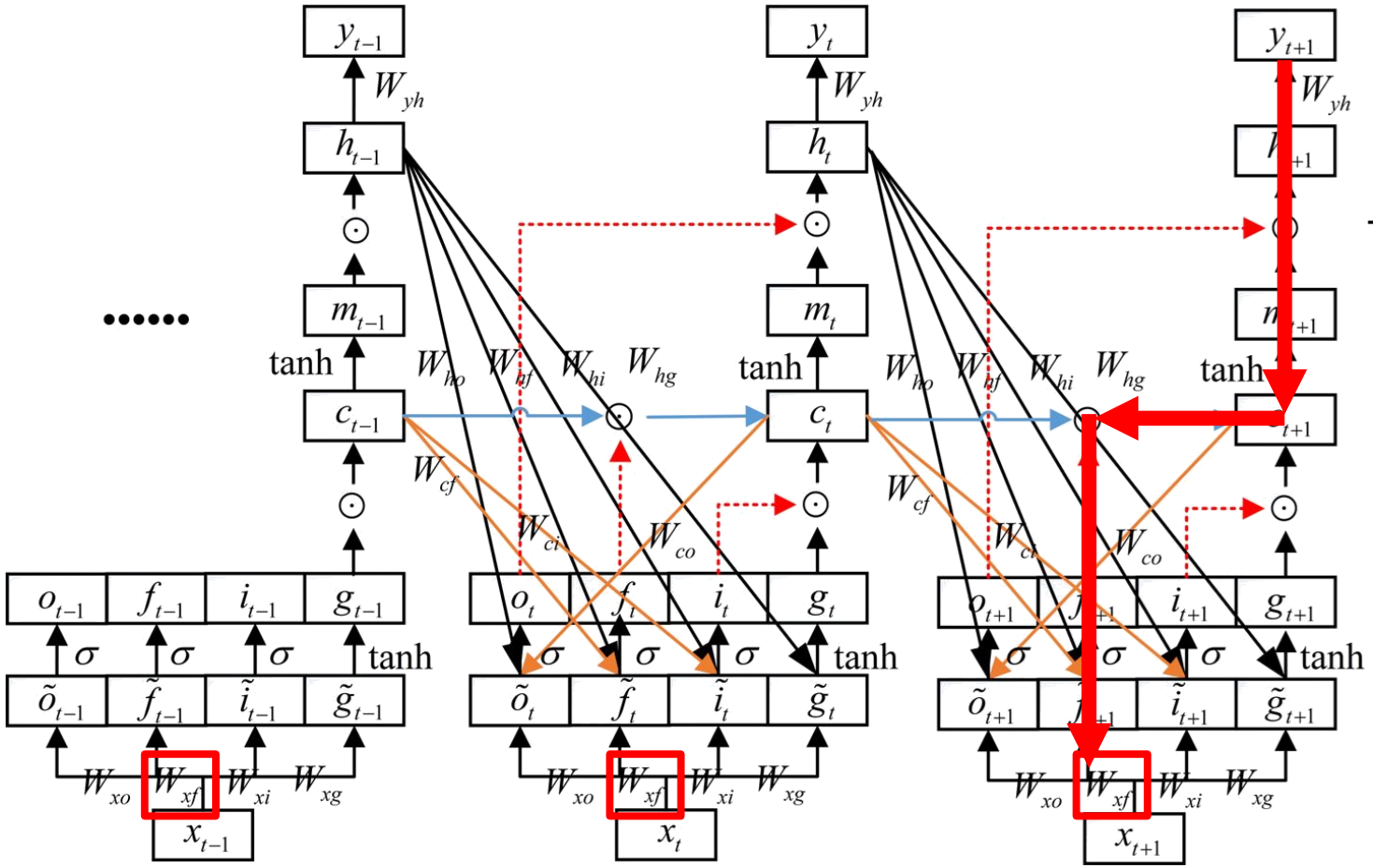
EXPLORATION: How LSTM Solves Vanishing Gradients

$t+1=3$



$$W_{hf}, W_{hg}, W_{hi}, W_{ho}, W_{xf}, W_{xg}, W_{xi}, W_{xo}, W_{yh}, b$$

EXPLORATION: How LSTM Solves Vanishing Gradients



L3

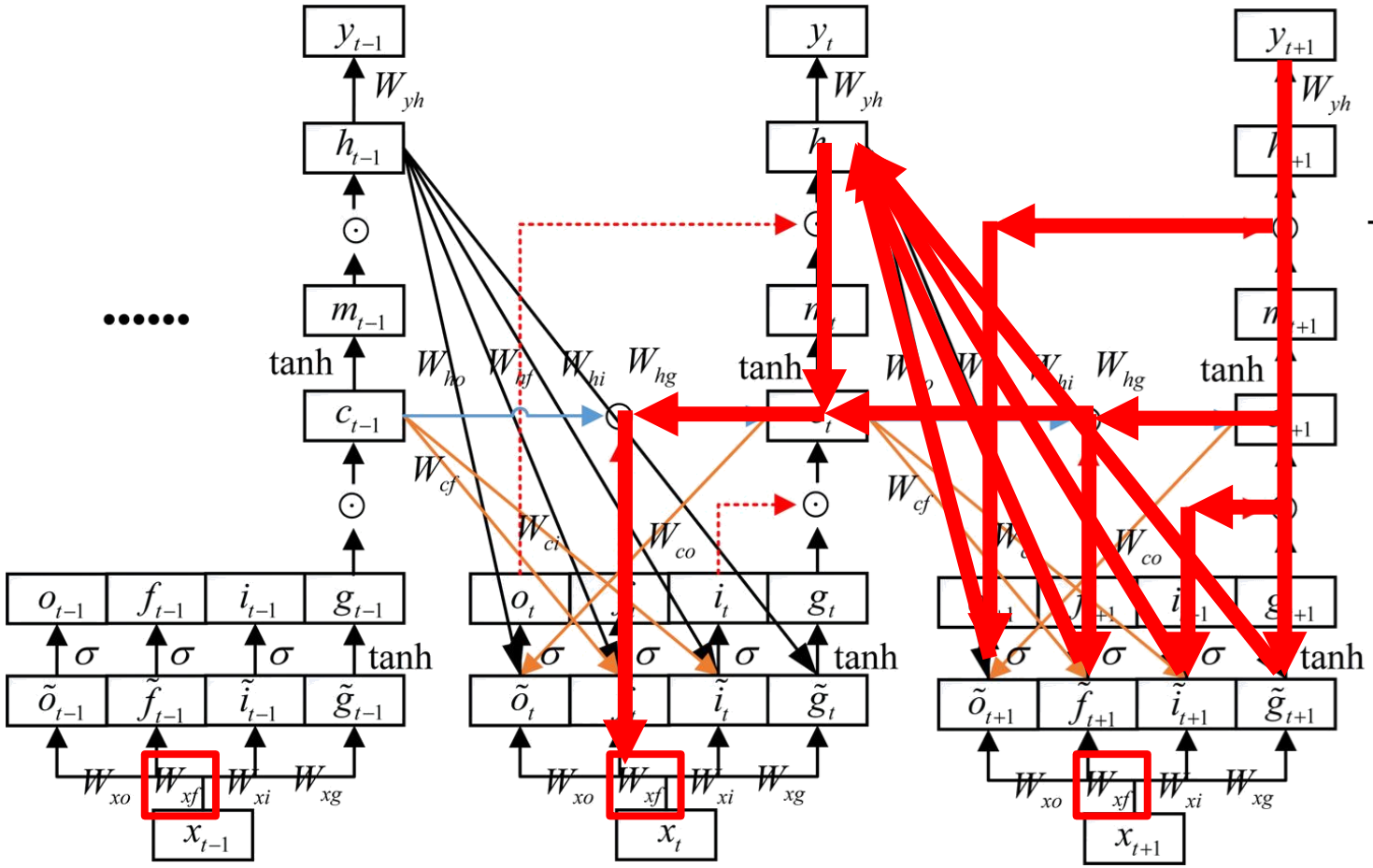
$$\frac{\partial L_3}{\partial W_{xf}} = \boxed{\frac{\partial L_3}{\partial W_{xf}^{(3)}}} + \frac{\partial L_3}{\partial W_{xf}^{(2)}} + \frac{\partial L_3}{\partial W_{xf}^{(1)}}$$

$$\frac{\partial L_3}{\partial W_{xf}^{(3)}} = \frac{\partial L_3}{\partial y_3}\frac{\partial y_3}{\partial h_3}\frac{\partial h_3}{\partial m_3}\frac{\partial m_3}{\partial c_3}\frac{\partial c_3}{\partial f_3}\frac{\partial f_3}{\partial \tilde{f}_3}\frac{\partial \tilde{f}_3}{\partial W_{xf}^{(3)}}$$

$$= \frac{\partial L_3}{\partial y_3}\frac{\partial y_3}{\partial h_3}\frac{\partial h_3}{\partial c_3}\frac{\partial c_3}{\partial f_3}\frac{\partial f_3}{\partial W_{xf}^{(3)}}$$

$$W_{hf}, W_{hg}, W_{hi}, W_{ho}, W_{xf}, W_{xg}, W_{xi}, W_{xo}, W_{yh}, b$$

EXPLORATION: How LSTM Solves Vanishing Gradients



L3

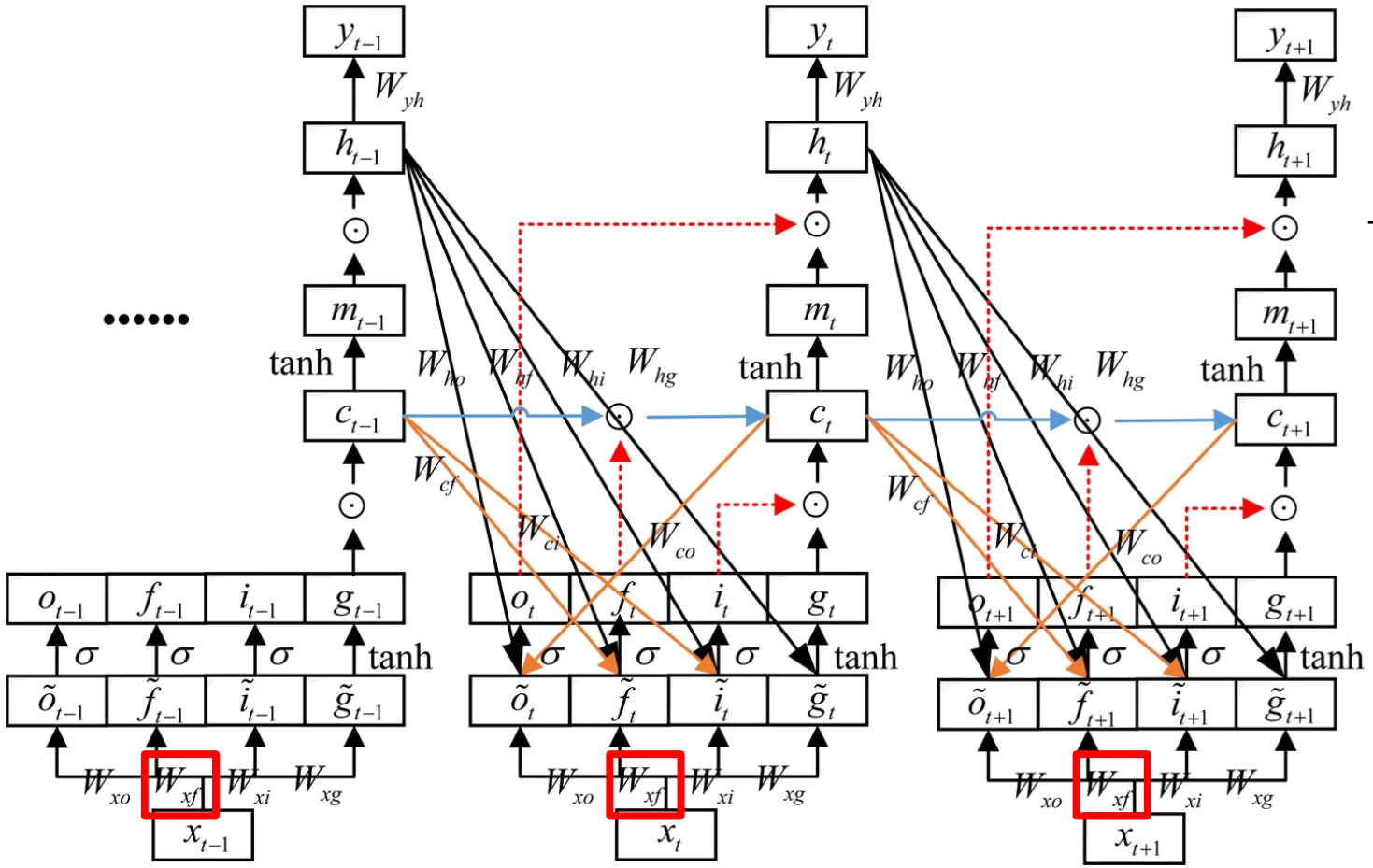
$$\frac{\partial L_3}{\partial W_{xf}} = \frac{\partial L_3}{\partial W_{xf}^{(3)}} + \boxed{\frac{\partial L_3}{\partial W_{xf}^{(2)}}} + \frac{\partial L_3}{\partial W_{xf}^{(1)}}$$

$$W_{hf}, W_{hg}, W_{hi}, W_{ho}, W_{xf}, W_{xg}, W_{xi}, W_{xo}, W_{yh}, b$$

EXPLORATION: How LSTM Solves Vanishing Gradients

$$\frac{\partial L_3}{\partial W_{xf}^{(2)}} = \frac{\partial L_3}{\partial y_3}\frac{\partial y_3}{\partial h_3}\left(\frac{\partial h_3}{\partial o_3}\frac{\partial o_3}{\partial h_2}\frac{\partial h_2}{\partial c_2} + \frac{\partial h_3}{\partial c_3}\frac{\partial c_3}{\partial c_2}\right)\frac{\partial c_2}{\partial f_2}\frac{\partial f_2}{\partial W_{xf}^{(2)}}$$

EXPLORATION: How LSTM Solves Vanishing Gradients



L3

$$\frac{\partial L_3}{\partial W_{xf}} = \frac{\partial L_3}{\partial W_{xf}^{(3)}} + \frac{\partial L_3}{\partial W_{xf}^{(2)}} + \boxed{\frac{\partial L_3}{\partial W_{xf}^{(1)}}}$$

$$W_{hf}, W_{hg}, W_{hi}, W_{ho}, W_{xf}, W_{xg}, W_{xi}, W_{xo}, W_{yh}, b$$

EXPLORATION: How LSTM Solves Vanishing Gradients

$$\frac{\partial L_3}{\partial W_{xf}^{(1)}} = \frac{\partial L_3}{\partial y_3}\frac{\partial y_3}{\partial h_3}\left(\left(\begin{array}{c} \frac{\partial h_3}{\partial o_3}\frac{\partial o_3}{\partial h_2} + \\ \frac{\partial h_3}{\partial c_3}\frac{\partial c_3}{\partial f_3}\frac{\partial f_3}{\partial h_2} + \\ \frac{\partial h_3}{\partial c_3}\frac{\partial c_3}{\partial i_3}\frac{\partial i_3}{\partial h_2} + \\ \frac{\partial h_3}{\partial c_3}\frac{\partial c_3}{\partial g_3}\frac{\partial g_3}{\partial h_2} \end{array}\right)\frac{\partial h_2}{\partial o_2}\frac{\partial h_1}{\partial c_1} + \left(\frac{\partial h_3}{\partial o_3}\frac{\partial o_3}{\partial h_2}\frac{\partial h_2}{\partial c_2}\frac{\partial c_2}{\partial c_1} + \frac{\partial h_3}{\partial c_3}\frac{\partial c_3}{\partial c_2}\frac{\partial c_2}{\partial c_1}\right)\right)\frac{\partial c_1}{\partial f_1}\frac{\partial f_1}{\partial W_{xf}^{(1)}}$$

EXPLORATION: How LSTM Solves Vanishing Gradients

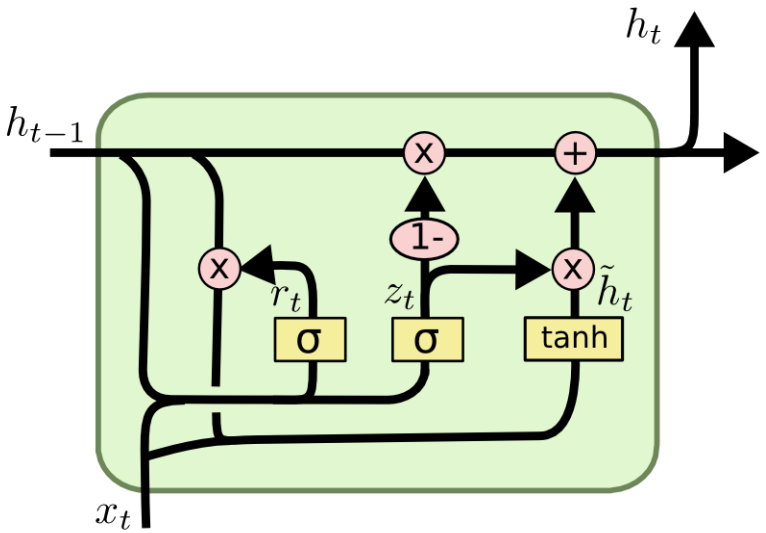
$$\text{where } \frac{\partial c_t}{\partial c_{t-1}} = \begin{array}{c} f_t \\ +c_{t-1}\sigma'(\cdot)W_{hf}O_{t-1}\tanh'(c_{t-1}) \\ +g_t\sigma'(\cdot)W_{hi}O_{t-1}\tanh'(c_{t-1}) \\ +i_t\sigma'(\cdot)W_{hg}O_{t-1}\tanh'(c_{t-1}) \end{array}$$

OUTLINE

This presentation is divided into 6 sections:

1. The Necessity: from RNN to LSTM
2. Structure and Function: Understand Core Idea
3. Variation: Gate Recurrent Unit (GRU) ☜
4. Application 1: Stock Market Forecast
5. Application 2: Seq2seq
6. Conclusion and Discussion
7. References

UNDERSTANDING GRU: Gated Recurrent Unit



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

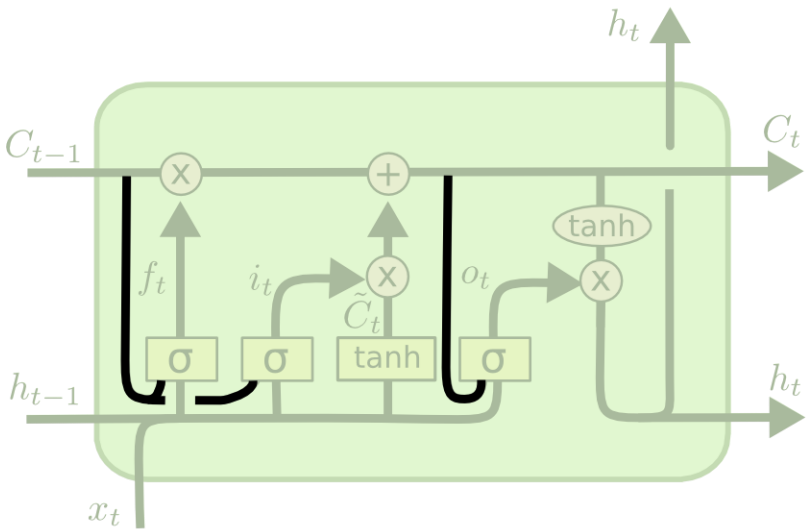
$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

GRU Structure

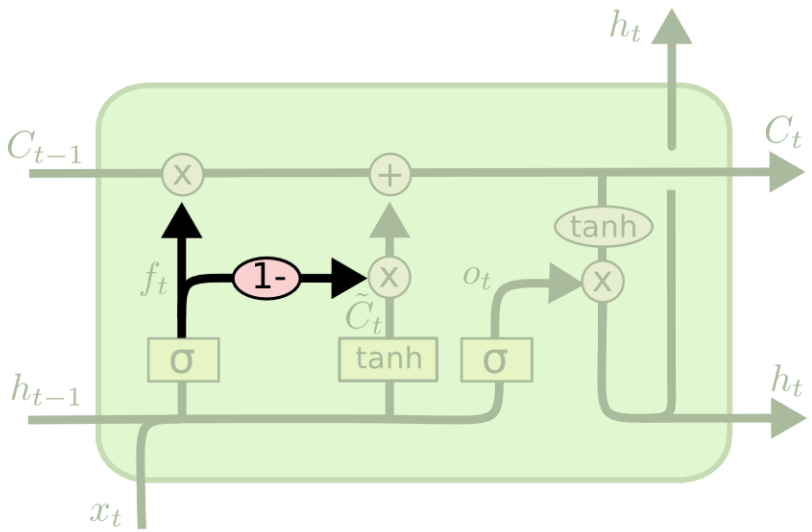
OTHER VARIANTS



$$f_t = \sigma\left(W_f \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \ + \ b_f\right)$$
$$i_t = \sigma\left(W_i \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \ + \ b_i\right)$$
$$o_t = \sigma\left(W_o \cdot [\boldsymbol{C_t}, h_{t-1}, x_t] \ + \ b_o\right)$$

Adding "Peephole Connections." This means that we let the gate layers look at the cell state.

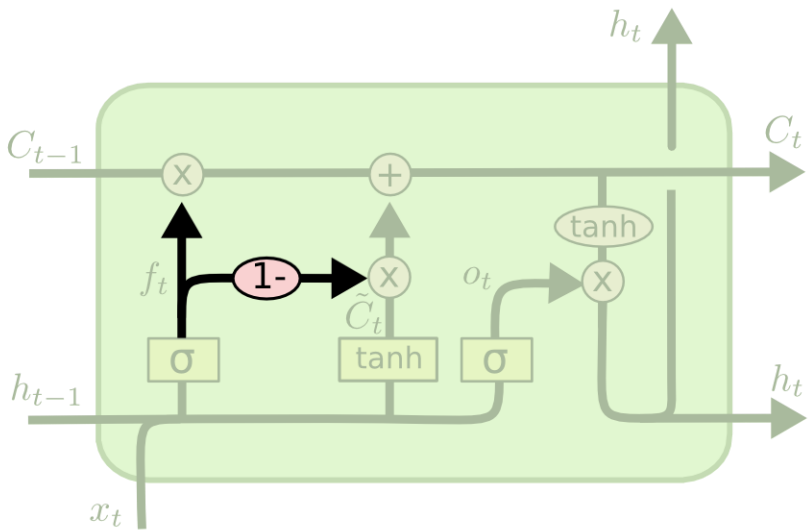
OTHER VARIANTS



$$C_t = f_t * C_{t-1} + (1 - \boldsymbol{f_t}) * \tilde{C}_t$$

To Use Coupled Forget and Input Gates

OTHER VARIANTS



$$C_t = f_t * C_{t-1} + (1 - \boldsymbol{f_t}) * \tilde{C}_t$$

To Use Coupled Forget and Input Gates

OUTLINE

This presentation is divided into 6 sections:

1. The Necessity: from RNN to LSTM
2. Structure and Function: Understand Core Idea
3. Variation: Gate Recurrent Unit (GRU)
4. Application 1: Stock Market Forecast ☜
5. Application 2: Seq2seq
6. Conclusion and Discussion
7. References

STOCK MARKET FORECAST WITH LSTM

STEP 1 Load Data

```
import tushare as ts
```

```
cons = ts.get_apis()
```

```
#获取沪深指数(000300)的信息，包括交易日期（datetime）、开盘价(open)、收盘价(close)，
#最高价(high)、最低价(low)、成交量(vol)、成交金额(amount)、涨跌幅(p_change)
df = ts.bar('000300', conn=cons, asset='INDEX', start_date='2010-01-01', end_date='')
```

```
df = df.dropna()
df.to_csv('sh300.csv')
```

STOCK MARKET FORECAST WITH LSTM

STEP 2 Check Data

```
df.columns
```

```
Index(['code', 'open', 'close', 'high', 'low', 'vol', 'amount', 'p_change'], dtype='object')
```

```
df.describe()
```

	open	close	high	low	vol	amount	p_change
count	2295.000000	2295.000000	2295.000000	2295.000000	2.295000e+03	2.295000e+03	2295.000000
mean	3100.514637	3103.181503	3128.213684	3073.658757	1.090221e+06	1.296155e+11	0.012397
std	627.888776	628.060844	634.870454	618.306225	9.284048e+05	1.268450e+11	1.483714
min	2079.870000	2086.970000	2118.790000	2023.170000	2.190120e+05	2.120044e+10	-8.750000
25%	2534.185000	2534.185000	2558.015000	2514.585000	5.634255e+05	6.092613e+10	-0.650000
50%	3160.800000	3165.910000	3193.820000	3134.380000	8.055400e+05	9.127102e+10	0.020000
75%	3484.665000	3486.080000	3510.940000	3461.215000	1.202926e+06	1.382787e+11	0.705000
max	5379.470000	5353.750000	5380.430000	5283.090000	6.864391e+06	9.494980e+11	6.710000

STOCK MARKET FORECAST WITH LSTM

STEP 3 Preprocess Data

```
#通过一个序列来生成一个31*(count(*)-train_end)矩阵（用于处理时序的数据）
#其中最后一列维标签数据。就是把当天的前n天作为参数，当天的数据作为label
def generate_data_by_n_days(series, n, index=False):
    if len(series) <= n:
        raise Exception("The Length of series is %d, while affect by (n=%d)." % (len(series), n))
    df = pd.DataFrame()
    for i in range(n):
        df['c%d' % i] = series.tolist()[i:-(n - i)]
    df['y'] = series.tolist()[n:]

    if index:
        df.index = series.index[n:]
    return df

#参数n与上相同。train_end表示的是后面多少个数据作为测试集。
def readData(column='high', n=30, all_too=True, index=False, train_end=-500):
    df = pd.read_csv("sh300.csv", index_col=0)
    #以日期为索引
    df.index = list(map(lambda x: datetime.datetime.strptime(x, "%Y-%m-%d"), df.index))
    #获取每天的最高价
    df_column = df[column].copy()
    #拆分为训练集和测试集
    df_column_train, df_column_test = df_column[:train_end], df_column[train_end - n:]
    #生成训练数据
```

STOCK MARKET FORECAST WITH LSTM

STEP 3 Preprocess Data

```
    df_generate_train = generate_data_by_n_days(df_column_train, n, index=index)
    if all_too:
        return df_generate_train, df_column, df.index.tolist()
    return df_generate_train
```

STOCK MARKET FORECAST WITH LSTM

STEP 4 Define Model

```
class RNN(nn.Module):
    def __init__(self, input_size):
        super(RNN, self).__init__()
        self.rnn = nn.LSTM(
            input_size=input_size,
            hidden_size=64,
            num_layers=1,
            batch_first=True
        )
        self.out = nn.Sequential(
            nn.Linear(64, 1)
        )

    def forward(self, x):
        r_out, (h_n, h_c) = self.rnn(x, None)  #None即隐层状态用0初始化
        out = self.out(r_out)
        return out
```

STOCK MARKET FORECAST WITH LSTM

STEP 5 Train the Model

```
from pandas.plotting import register_matplotlib_converters
register_matplotlib_converters()
# 获取训练数据、原始数据、索引等信息
df, df_all, df_index = readData('high', n=n, train_end=train_end)

#可视化原高价数据
df_all = np.array(df_all.tolist())
plt.plot(df_index, df_all, label='real-data')
plt.legend(loc='upper right')


#对数据进行预处理，规范化及转换为Tensor
df_numpy = np.array(df)

df_numpy_mean = np.mean(df_numpy)
df_numpy_std = np.std(df_numpy)

df_numpy = (df_numpy - df_numpy_mean) / df_numpy_std
df_tensor = torch.Tensor(df_numpy)


trainset = mytrainset(df_tensor)
trainloader = DataLoader(trainset, batch_size=batch_size, shuffle=False)
```

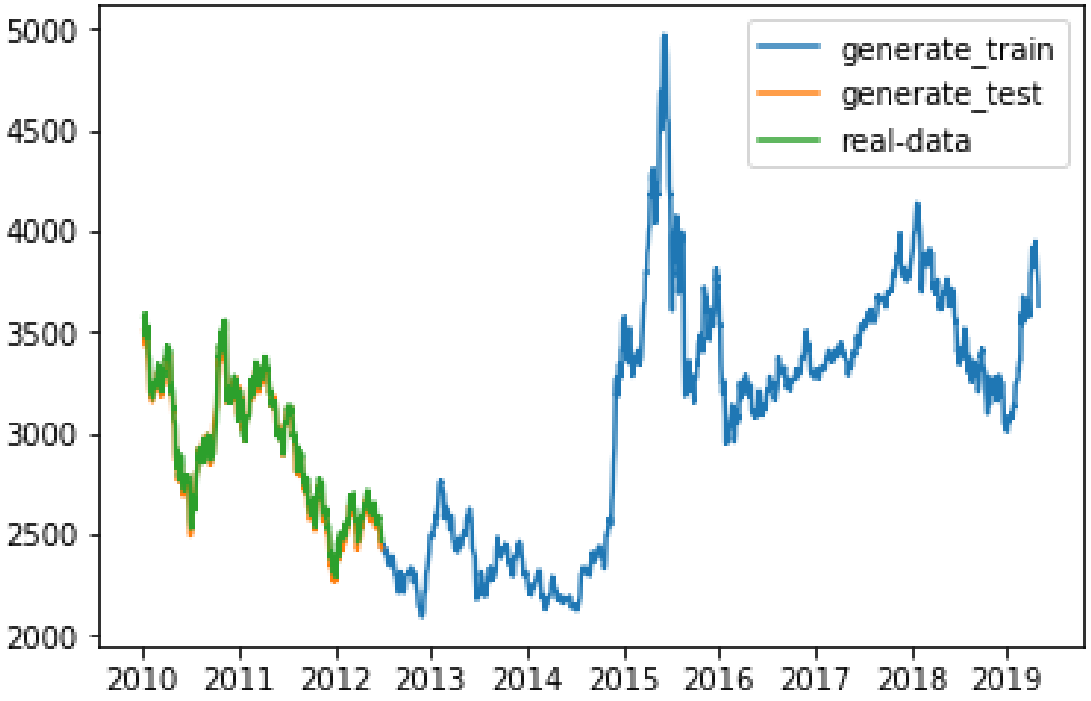
STOCK MARKET FORECAST WITH LSTM

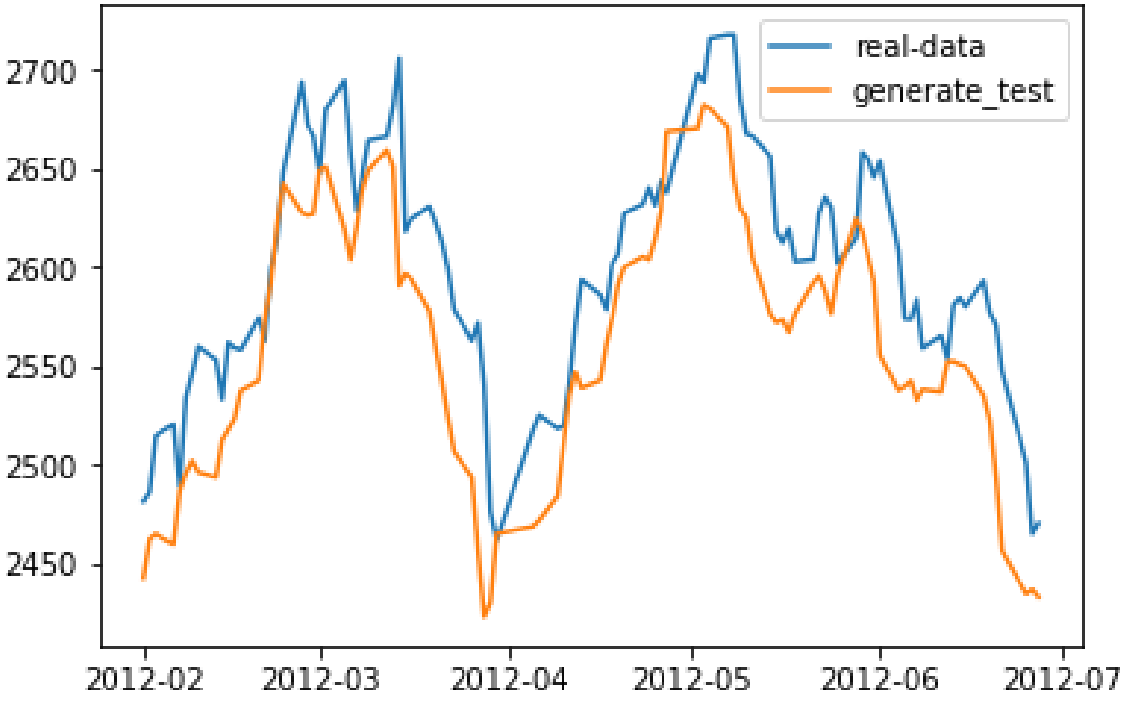
STEP 6 Test the Model

```
generate_data_train = []
generate_data_test = []

test_index = len(df_all) + train_end

df_all_normal = (df_all - df_numpy_mean) / df_numpy_std
df_all_normal_tensor = torch.Tensor(df_all_normal)
for i in range(n, len(df_all)):
    x = df_all_normal_tensor[i - n:i].to(device)
    #rnn的输入必须是3维，故需添加两个1维的维度，最后成为[1,1,input_size]
    x = torch.unsqueeze(torch.unsqueeze(x, dim=0), dim=0)

    y = rnn(x).to(device)
    if i < test_index:
        generate_data_train.append(torch.squeeze(y).detach().cpu().numpy() * df_numpy_std + df_numpy_mean)
    else:
        generate_data_test.append(torch.squeeze(y).detach().cpu().numpy() * df_numpy_std + df_numpy_mean)
plt.plot(df_index[n:train_end], generate_data_train, label='generate_train')
plt.plot(df_index[train_end:], generate_data_test, label='generate_test')
plt.plot(df_index[train_end:], df_all[train_end:], label='real-data')
plt.legend()
plt.show()
```

STOCK MARKET FORECAST WITH LSTM

STEP 6 Test the Model



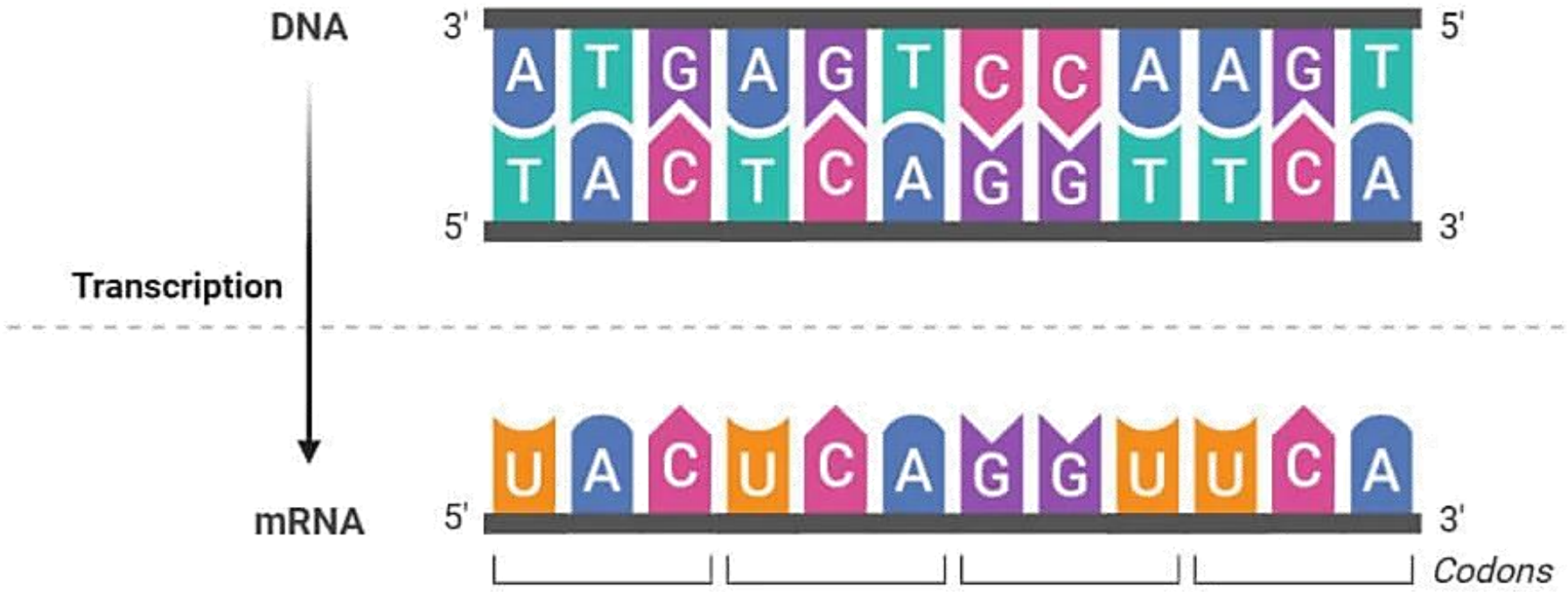


OUTLINE

This presentation is divided into 6 sections:

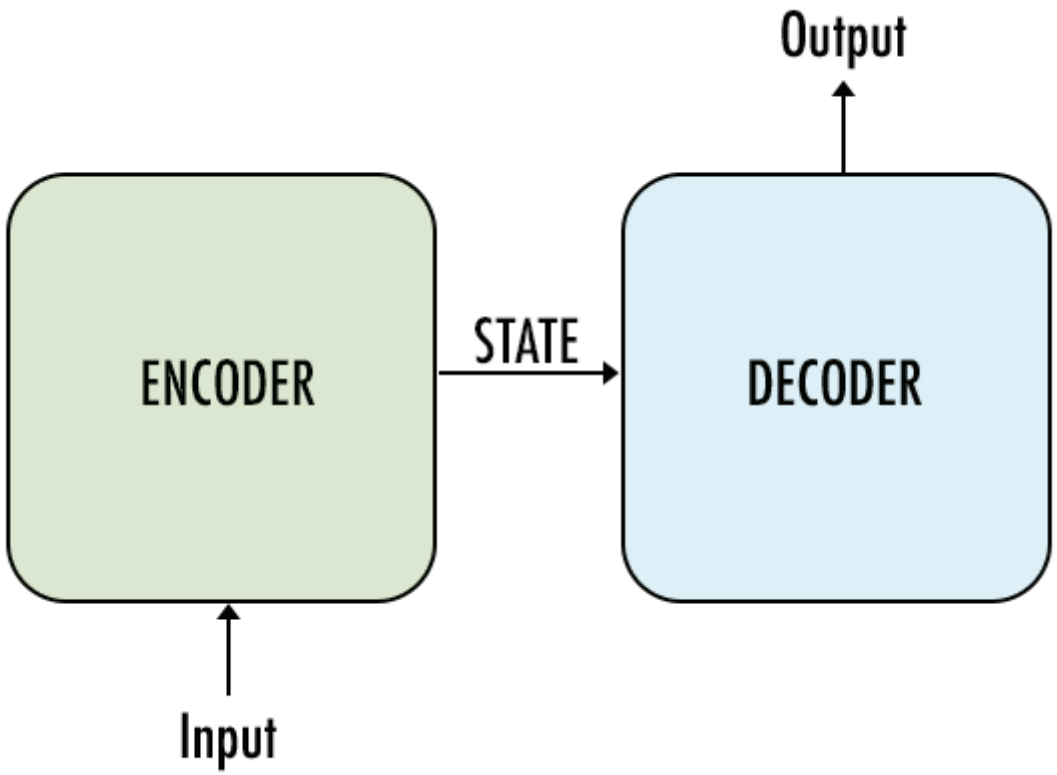
1. The Necessity: from RNN to LSTM
2. Structure and Function: Understand Core Idea
3. Variation: Gate Recurrent Unit (GRU)
4. Application 1: Stock Market Forecast
5. Application 2: Seq2seq ☜
6. Conclusion and Discussion
7. References

SEQ2SEQ: Sequence to Sequence

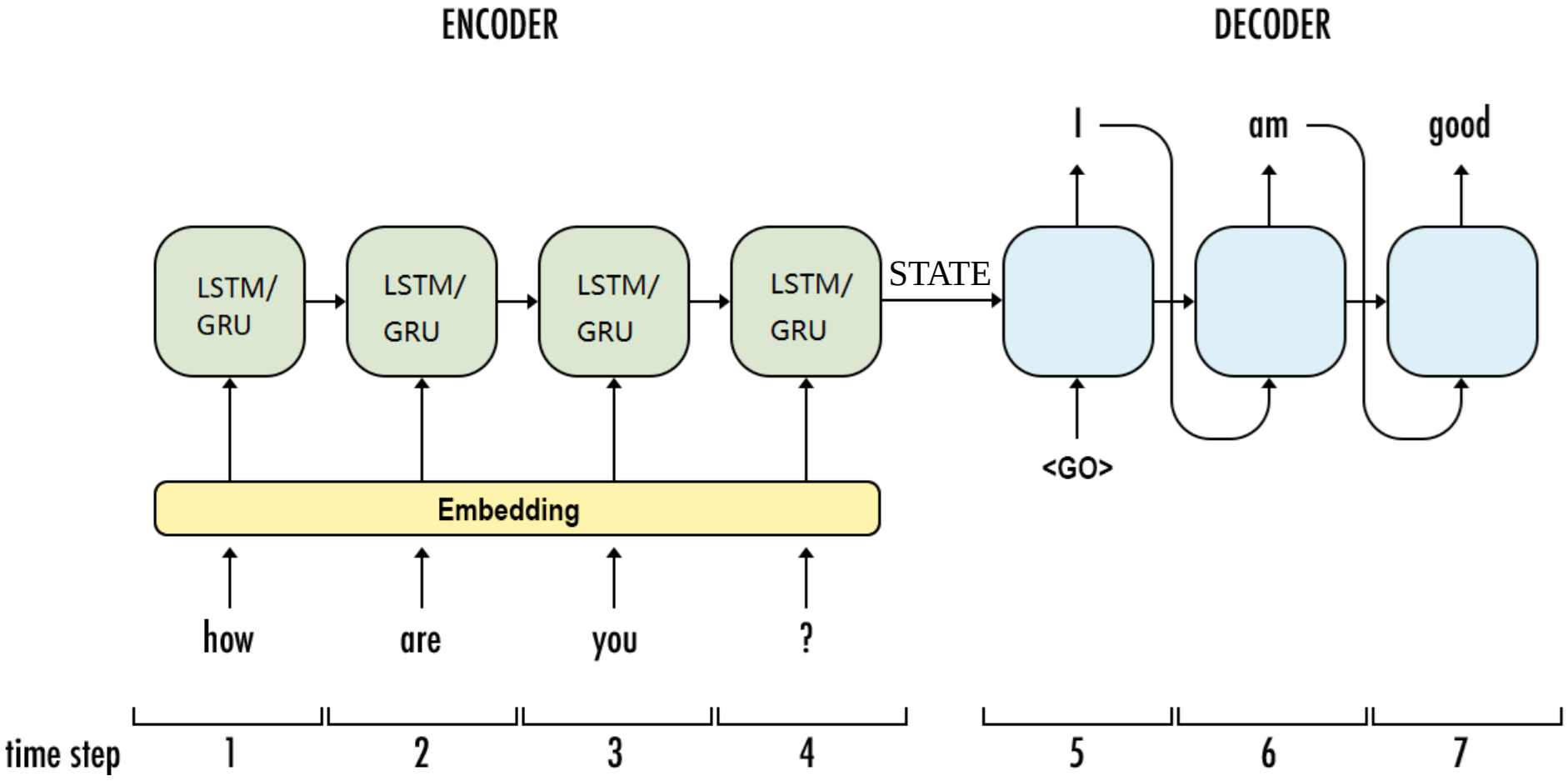


Transcription: from DNA to RNA

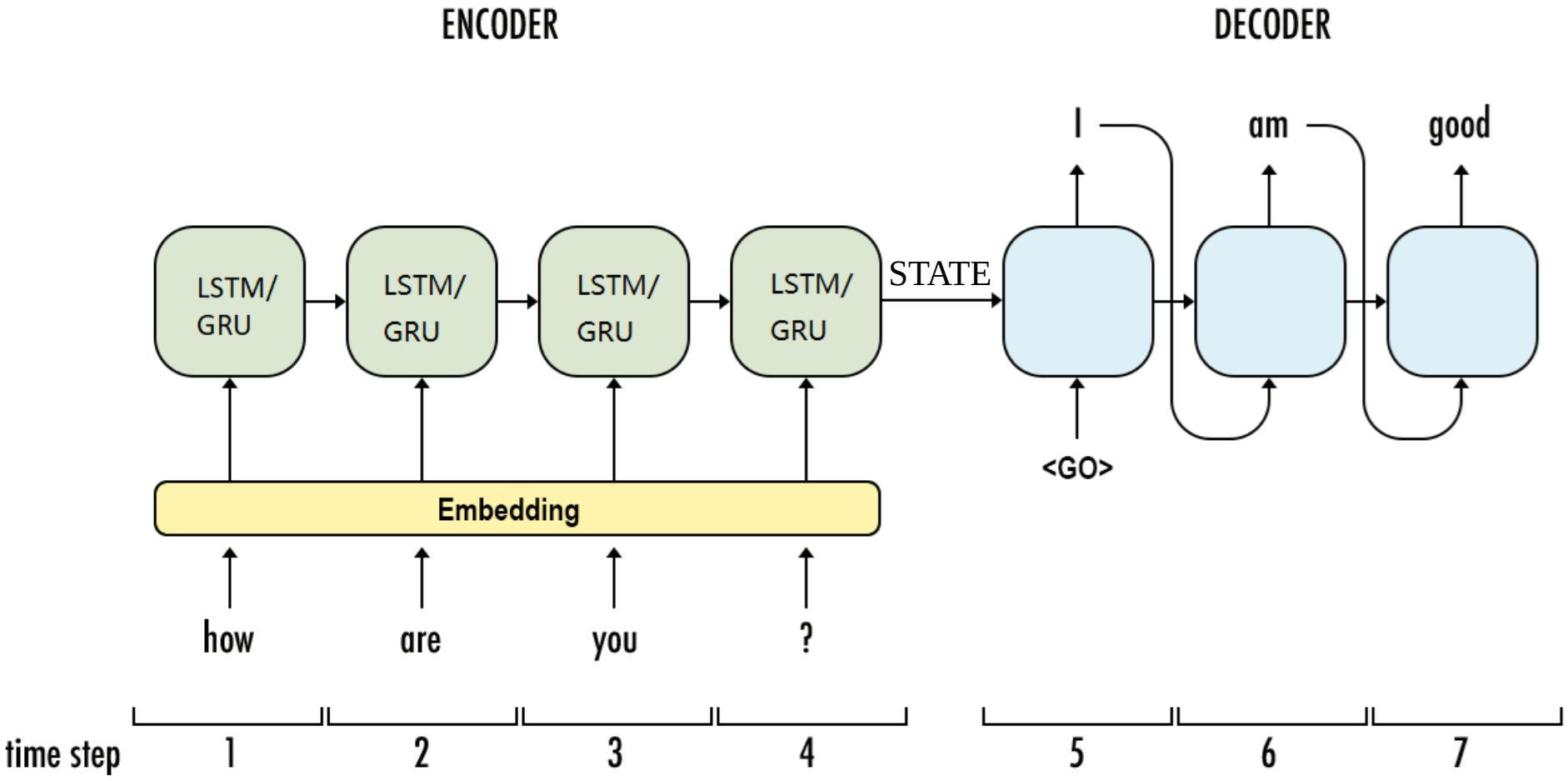
SEQ2SEQ: Sequence to Sequence



SEQ2SEQ: Sequence to Sequence



SEQ2SEQ: Sequence to Sequence



YARDSTICK: Bilingual Evaluation Understudy (BLEU)

p_n merges the accuracy information provided by all n-gram.

E.g. The label is `*A B C D E F*`, while the prediction is `*A B B C D*` , we have:

$$p_1 = \frac{4}{5}, \quad p_2 = \frac{3}{4}, \quad p_3 = \frac{1}{3}, \quad p_4 = 0.$$

DEFINITION (BLEU):

$$e^{\left(\min\left(0, 1 - \frac{\text{len(label)}}{\text{len(prediction)}}\right)\right)} \prod_{n=1}^{k} p_n^{\frac{1}{2^n}}$$

punish too short prediction

long matches have larger weights

OUTLINE

This presentation is divided into 6 sections:

1. The Necessity: from RNN to LSTM
2. Structure and Function: Understand Core Idea
3. Variation: Gate Recurrent Unit (GRU)
4. Application 1: Stock Market Forecast
5. Application 2: Seq2seq
6. Conclusion and Discussion ☜
7. References

CONCLUSION & DISCUSSION

OUTLINE

This presentation is divided into 6 sections:

1. The Necessity: from RNN to LSTM
2. Structure and Function: Understand Core Idea
3. Variation: Gate Recurrent Unit (GRU)
4. Application 1: Stock Market Forecast
5. Application 2: Seq2seq
6. Conclusion and Discussion
7. References ☜

REFERENCES (F.Y.I.)

[1] Olah C. Understanding lstm networks[J]. 2015.

[2] Goyal P, Pandey S, Jain K. Deep learning for natural language processing[J]. New York: Apress, 2018.

[3]吴茂贵, 王红星, 刘未昕, 等. Python 入门到人工智能实战[M]. BEIJING BOOK CO. INC., 2020.

[4] Yu Y, Si X, Hu C, et al. A review of recurrent neural networks: LSTM cells and network architectures[J]. Neural computation, 2019, 31(7): 1235-1270.

[5] Karpathy A. The unreasonable effectiveness of recurrent neural networks, 2015[J]. URL http://karpathy. github. io/2015/05/21/rnn-effectiveness, 2016.

[6] Hochreiter S, Schmidhuber J. Long short-term memory[J]. Neural computation, 1997, 9(8): 1735-1780.

THANK YOU!

XU Chentianye · 16 Oct, 2021